

國立臺灣大學電機資訊學院電信工程學研究所

碩士論文

Graduate Institute of Communication Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

基於遞迴最小平方法之實時神經視訊編碼碼率控制

Real-Time Rate Control for Neural Video Coding via
Recursive Least Squares

陳祈安

Chi-An Chen

指導教授：丁建均 博士

Advisor: Jian-Jiun Ding, Ph.D.

中華民國 115 年 1 月

January 2026



誌謝



感謝我的指導老師丁建均教授，在我渾渾噩噩、語無倫次時，耐心、循循善誘地引導我一步步往前進。從大學時就受老師的照顧，一直到研究所畢業，三年多的時間，在老師的栽培下，總算長成在各方面都更有應對策略的人，除了研究、學術上的提點，在待人接物、人生閱歷的分享上，老師總是用溫和且真誠的態度面對、指引我們，真心感謝。

碩士的生活像是從池塘被拋到大海，在越來越沒有拘束的世界浮浮沉沉，事情開始不那麼簡單的被框架，沒有起跑和終點線，也許每個時代都一樣，也或許現今的時代真的有幾分不同，我似乎老是在漂流，被資訊的海嘯推來推去，難以靜下心來專注，但無數懷疑自己的晚上，太陽都還是會升起。

人生裡幸運的事情太多了，回首來時路，似乎每件事情都被溫柔的安排，給了我那麼多財富、那麼多真心相伴的人。感謝我的父母、撒嬌的貓，你們是那麼的好，讓我有無限的動力持續努力；感謝碩士期間新認識的所有朋友，不管是實驗室的夥伴還是實習的主管同事，都讓我感覺不用去偽裝，可以自在的相處。

人生的階段來來去去，碩士畢業也許是最後一個這麼標準的階段句號，往後的生活是更綿延不斷地漂泊，但願相伴過的大家在人生的旅途都能找到自己的平衡、心靈的平靜，期待和每個人再次相遇時，我們身上的變化。

摘要



神經視訊編碼 (Neural Video Coding, NVC) 近年來在壓縮效率上已展現出超越傳統視訊編碼標準 (如 H.265/HEVC, H.266/VVC) 的潛力。然而，如何將其實現於實時應用 (如視訊會議、直播) 仍是一大挑戰。儘管 DCVC-RT 等高效能架構已突破了計算複雜度的瓶頸，但目前針對此類先進 NVC 模型的實時碼率控制 (Rate Control) 機制研究仍然匱乏，且現有方法難以適應神經網路特有的非線性與非穩態特徵。

本論文提出了一種針對實時神經視訊編碼器 DCVC-RT 的自適應狀態空間碼率控制系統。首先，透過全域搜索分析，本研究確立了 DCVC-RT 的碼率與量化參數 (QP) 之間呈現對數關係 (Logarithmic R-Q Model)，並利用此線性化特性引入遞迴最小平方法 (Recursive Least Squares, RLS) 進行在線參數估計。相較於傳統梯度下降法，RLS 演算法具備極快的收斂速度與穩定性。針對 DCVC-RT 特有的週期性特徵刷新機制，本研究進一步提出了「雙狀態 RLS (Dual-State RLS)」策略，將一般幀與刷新幀的參數估計分離，有效解決了刷新幀作為異常值導致的模型參數劇烈震盪 (Model Shattering) 問題。此外，本系統結合了滑動視窗預算分配、層級化質量結構以及智慧輸出平滑化機制，以確保長期流量的穩定性與視覺品質。

實驗結果顯示，在 UVG、MCL-JCV 與 HEVC Class B 等多個標準數據集上，本研究提出的方法均優於現有的基準方法。特別是在 UVG 數據集上，相比於基準算法，本方法實現了平均 14.9% 的 BD-Rate 性能提升，並將平均碼率誤差 (BRE) 控制在 1.92% 的高精準度範圍內。同時，計算複雜度分析顯示本方法的額外時間開銷僅約 1%，證實了其在實時應用中的可行性。本研究成功填補了高效能 NVC 與實時傳輸需求之間的缺口，為神經視訊編碼的實際部署提供了關鍵的技術解決方案。

關鍵字：

神經視訊編碼、碼率控制、遞迴最小平方法、實時視訊通訊、DCVC-RT

ABSTRACT



Neural Video Coding (NVC) shows great potential to surpass traditional standards like H.265/HEVC and H.266/VVC in compression efficiency. However, realizing NVC in real-time applications remains challenging. While architectures like DCVC-RT have reduced computational complexity, effective real-time Rate Control (RC) mechanisms for these models are scarce, as existing methods struggle with the non-linear and non-stationary nature of neural networks.

This thesis proposes an adaptive state-space RC system specifically designed for the real-time neural video coder, DCVC-RT. Global search analysis reveals a logarithmic relationship between bitrate and the Quantization Parameter (QP) in DCVC-RT. Leveraging this linearity, a Recursive Least Squares (RLS) algorithm is introduced for online parameter estimation, offering superior convergence and stability over gradient-based methods. To address the periodic feature refresh mechanism in DCVC-RT, a "Dual-State RLS" strategy is proposed. By separating parameter estimation for normal and refresh frames, this strategy prevents model shattering caused by outliers. Furthermore, the system integrates sliding window budget allocation, a hierarchical quality structure, and smart output smoothing to ensure traffic stability and visual quality.

Experimental results on UVG, MCL-JCV, and HEVC Class B datasets demonstrate that the proposed method significantly outperforms baselines. On the UVG dataset, it achieves a 14.9% BD-Rate improvement and maintains an average Bitrate Error (BRE) of 1.92%. With approximately 1% additional time overhead, the method proves feasible for real-time use. This study bridges the gap between high-performance NVC and real-time transmission needs, offering a key solution for practical deployment.

Keywords:

Neural Video Coding, Rate Control, Recursive Least Squares, Real-Time Video Communication, DCVC-RT

CONTENTS



摘要

ABSTRACT

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1 Introduction

1.1	Background	1
1.2	Problem Statement	2
1.3	Motivation	3
1.4	Primary Contributions	4
1.5	Thesis Organization	5

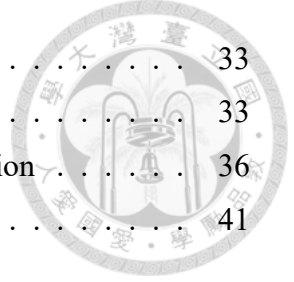
Chapter 2 Fundamentals

2.1	Video Compression Basics	7
2.1.1	Information Theory – Entropy	7
2.1.2	Redundancy	8
2.1.3	The Rate-Distortion (R-D) Trade-off	11
2.2	From Traditional Standards to Neural Video Coding	13
2.2.1	The Generalized Hybrid Coding Pipeline	13
2.2.2	Temporal Prediction: Motion Vectors to Optical Flow	15
2.2.3	Spatial Transformation: Linear to Non-linear	16
2.2.4	Differentiable Quantization and Optimization	17
2.2.5	Entropy Coding: Context Models to Learned Priors	19

Chapter 3 Literature Review

3.1	State-of-the-Art Neural Video Codecs	23
3.1.1	Developmental Progression	23
3.1.2	Conditional Coding and DCVC Architecture	24
3.1.3	Single Model, Variable R-D Performance	27
3.1.4	Wider Quality Range and Real-Time Adaptation	30
3.1.5	Summary	32

3.2	Rate Control Algorithms and Limitations	33
3.2.1	Rate Control in Traditional Codecs	33
3.2.2	Rate Control Approaches in Neural Video Compression	36
3.2.3	Comparative Analysis and Limitations	41



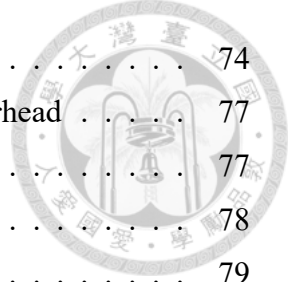
Chapter 4 Proposed Method: Adaptive State-Space Design for Real-Time

NVC Rate Control	47
4.1	System Overview 47
4.2	R-Q Modeling and Empirical Analysis 49
4.3	Adaptive Parameter Estimation 53
4.3.1	Recursive Least Squares (RLS) Algorithm 53
4.3.2	Comparison with Gradient-based Methods 55
4.3.3	Challenge: Vulnerability to Refresh Frames 56
4.3.4	Proposed Solution: Dual-State RLS 57
4.3.5	Parameter Stabilization (First-Stage Smoothing) 59
4.4	Bitrate Allocation Strategy 60
4.4.1	GOP-based Allocation with Sliding Window 60
4.4.2	Hierarchical Quality Structure and Feature Propagation 60
4.4.3	Dynamic GOP Rebalancing 61
4.5	Dynamic Smoothing and Control 62
4.5.1	Smart Output Smoothing (Second-Stage Smoothing) 62
4.5.2	Algorithm Summary 63

Chapter 5 Experiments and Results

5.1	Experimental Setup 65
5.1.1	Datasets 65
5.2	Evaluation Benchmarks 67
5.2.1	Rate Control Accuracy 67
5.2.2	R-D Performance 67
5.2.3	Time Overhead 68
5.3	Implementation Details 69
5.3.1	Experimental Settings 69
5.3.2	Baseline Methods 69
5.3.3	Parameter Settings 71
5.4	Performance Comparison 73
5.4.1	Quantitative Results on UVG Dataset 73

5.4.2	Generalization on Diverse Datasets	74
5.4.3	Environment Specifications and Computational Overhead	77
5.5	Parametric Analysis and Robustness	77
5.5.1	Robustness to Initialization	78
5.5.2	Sensitivity to Update Dynamics	79
5.6	Summary	80
Chapter 6	Conclusion	83
6.1	Summary of Contributions	83
6.2	Limitations and Future Work	84
	References	85



LIST OF FIGURES

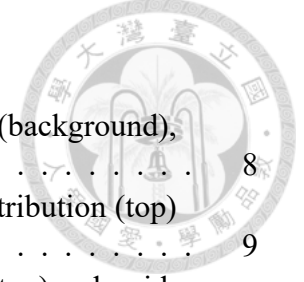


Figure 2.1	Spatial redundancy in the 'Astronaut' image: flat (background), edge (facial features), and texture (hair) regions.	8
Figure 2.2	Statistical comparison: original pixel intensities distribution (top) vs. prediction residuals distribution (bottom).	9
Figure 2.3	Temporal redundancy reduction: motion estimation (top) and residual comparison (bottom).	10
Figure 2.4	The mechanics of quantization. Coarse quantization (bottom path) reduces bitrate at the cost of reconstruction fidelity.	12
Figure 2.5	Rate-Distortion (R-D) performance curves evaluated on the UVG dataset, illustrating the trade-off between quality (PSNR) and bitrate (bpp).	13
Figure 2.6	Comparison between the traditional predictive coding architecture (left) and the end-to-end Deep Video Compression (DVC) framework (right) [18].	14
Figure 2.7	The motion processing pipeline in DVC.	16
Figure 2.8	Visual comparison of spatial energy distribution. The NVC model [23] extracts semantic features (c) using CompressAI [24], unlike the rigid blocks in traditional DCT (b).	16
Figure 2.9	Quantization strategies in NVC. Left: The optimization landscape defined by λ . Right: To resolve the zero-gradient problem of hard quantization (dots), uniform noise (dashed line) is added during training as a differentiable proxy [25].	18
Figure 2.10	Illustration of Binary Arithmetic Coding.	20
Figure 2.11	Conceptual comparison of Entropy Models [27].	21
Figure 2.12	Operational diagram of the Hyperprior architecture. The input y is analyzed by the hyper-branch (top) to generate compressed side-information $\hat{z}$. The reconstructed $\hat{z}$ is then decoded to predict the mean and scales (μ, σ) , which model the statistical distribution of $\hat{y}$ for efficient arithmetic coding.	22
Figure 3.1	The paradigm shift from a residue coding-based framework to a conditional coding-based framework [44].	25
Figure 3.2	Architecture of the temporal context mining (TCM) module [45].	26
Figure 3.3	Multi-granularity quantization and the corresponding inverse quantization [48].	27
Figure 3.4	Entropy coding with quadtree partition. It fully mines the correlation from both spatial and channel dimensions [51].	28
Figure 3.5	Hierarchical quality structure comparison [51].	29
Figure 3.6	The Feature Modulation mechanism in DCVC-FM [53].	30
Figure 3.7	Rate-distortion performance comparison [53].	31
Figure 3.8	Comparison between (a) the traditional motion-based NVC paradigm and (b) the proposed DCVC-RT architecture. The shift to implicit modeling and single-scale latents significantly reduces operational complexity [8].	32
Figure 3.9	The framework of the Rate-Quality ($R - Q_s$) based rate control scheme proposed by [11].	39

Figure 3.10 Overview of the fully neural rate control system proposed by Zhang et al. [12]. (a) The framework utilizes a Weight Estimation Network for hierarchical allocation and a Rate Implementation Network for parameter prediction. (b) Visualization of the hierarchical allocation strategy, distributing bits from miniGoP level down to frame level based on feature importance.	40
Figure 4.1 Overall architecture of the proposed rate control system. The DCVC-RT is treated as a black-box model. The controller determines the QP_{final} based on the target bitrate and updates its internal model using the feedback loop after encoding.	47
Figure 4.2 Empirical R-Q curve fitting based on exhaustive QP search data. The blue curve (Logarithmic model) demonstrates the highest fidelity to the actual data points across the entire bitrate range compared to other candidates.	50
Figure 4.3 Comparison of parameter evolution between Power Model (Left) and Log Model (Right). Top row: <i>Beauty</i> (Low SI/TI); Bottom row: <i>ReadySteadyGo</i> (High TI). The Log Model parameters (α) exhibit stable periodicity within a predictable range ($10 \sim 20$), whereas the Power Model parameters fluctuate wildly ($100 \sim 550$) and lack clear patterns.	51
Figure 4.4 Internal mechanism of the RLS Estimator. The model updates its parameter state θ and covariance matrix $\mathbf{P}$ recursively based on the prediction error of the previous frame.	53
Figure 4.5 The internal structure of the temporal context module ($f_{Tcontext}$). The Refresh Control determines whether the context is generated from the propagated feature F_{t-1} (Standard Mode) or the reconstructed image $\hat{x}_{t-1}$ (Refresh Mode). This path switching fundamentally alters the signal characteristics.	56
Figure 4.6 The "Model Shattering" phenomenon caused by single-state RLS. Due to the massive outliers introduced by Refresh Frames, the estimator's covariance matrix is irreversibly corrupted. This causes the model parameters (Alpha and Beta) to collapse towards zero and fail to recover, losing their ability to track the true R-Q characteristic of standard frames.	58
Figure 4.7 Performance of the Dual-State RLS. By decoupling the refresh frames, the parameters maintain stability without collapsing to zero. The curves show stable oscillations, confirming that the estimator correctly distinguishes between the two encoding modes.	59
Figure 5.1 Examples of sequences containing hard cuts (editorial shot transitions). To align with the continuous scene characteristic of real-time streaming, sequences such as <i>Kimono</i> from HEVC Class B and several clips from MCL-JCV were excluded from our evaluation set.	66
Figure 5.2 Rate-Distortion performance on the UVG dataset. The red dotted lines illustrate our target rate selection strategy: four distinct bitrates are determined based on the anchor QP settings (5, 20, 35, 50) to evaluate the controller's performance across different bandwidth constraints.	70

LIST OF TABLES

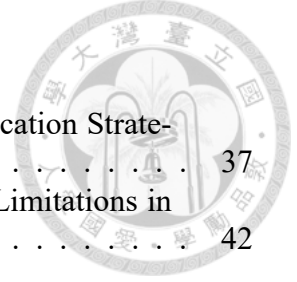


Table 3.1	Comparison of Rate Control Schemes: Models, Allocation Strategies, and Updating Mechanisms.	37
Table 3.2	Comparison of Experimental Settings and Practical Limitations in Existing Studies.	42
Table 4.1	Comparison between Gradient-based (LMS/SGD) algorithms and the proposed RLS Filter for Rate Control.	55
Table 5.1	Parameter settings for Baseline-Lambda.	71
Table 5.2	Parameter settings for Baseline 2022p.	72
Table 5.3	Parameter settings for Baseline 2024.	72
Table 5.4	Parameter settings for Proposed-Iter.	72
Table 5.5	Parameter settings for Proposed-RLS.	73
Table 5.6	Comparison of Rate Control Performance on UVG Dataset. Metrics include Bitrate Error (BRE %) and BD-Rate (PSNR). Bold indicates the best performance, and <u>underline</u> indicates the second best. (Note: Lower BRE indicates better accuracy; more negative BD-Rate indicates better coding efficiency.)	74
Table 5.7	Rate Control Performance on the MCL-JCV Dataset. Bold indicates best, <u>underline</u> indicates second best. (Note: Lower BRE is better; more negative BD-Rate is better.)	75
Table 5.8	Rate Control Performance on the HEVC Class B Dataset. (Note: Lower BRE is better; more negative BD-Rate is better.)	75
Table 5.9	Comparison of Time Overhead relative to the Rule-Based method.	77
Table 5.10	Ablation Study on Initialization Parameter (Proposed-RLS). Bold indicates the best performance. (Note: Robustness is shown by maintaining low BRE and negative BD-Rate across settings.)	78
Table 5.11	Ablation Study on Initialization Parameter (Baseline-Lambda). (Note: High BRE and positive BD-Rate indicate model divergence.)	79
Table 5.12	Ablation Study on Forgetting Factor λ (Proposed-RLS). Bold indicates the best performance. (Note: Consistently negative BD-Rate indicates high stability.)	80
Table 5.13	Ablation Study on Learning Rate (Baseline-Lambda). Bold indicates the best performance. (Note: Large positive BD-Rate values at Lr0.5 indicate catastrophic failure.)	80

Chapter 1

Introduction



1.1 Background

Modern digital life and network communication are fundamentally built upon efficient data compression technologies. With the explosive growth of global data volume—particularly the rapid increase in multimedia content—compression techniques have become increasingly critical for maintaining Quality of Service (QoS).

Video content has now become the dominant source of network traffic. According to the Ericsson Mobility Report [1], video traffic is projected to account for approximately 76% of all mobile data traffic by the end of 2025. This trend is driven not only by traditional video streaming platforms such as YouTube, Netflix, and TikTok, but has also been further intensified by the widespread adoption of short-form video sharing on social media, real-time video conferencing, and the growing prevalence of high-quality images and videos generated by artificial intelligence (AI). Consequently, designing high-performance video compression systems that can deliver higher visual quality under limited bandwidth budgets has become an increasingly urgent challenge.

Although conventional video coding standards—such as H.264/AVC [2], H.265/HEVC [3], and H.266/Versatile Video Coding (VVC) [4]—have achieved remarkable success and have been continuously optimized over the past decades, Neural Video Coding (NVC) has emerged in recent years as a promising alternative. By leveraging the strong nonlinear modeling capability of neural networks, NVC has demonstrated significant potential to surpass traditional hybrid video coding frameworks in terms of compression

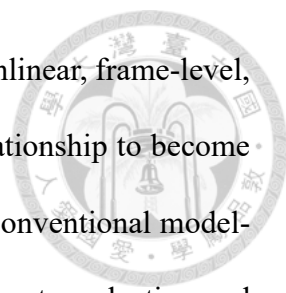
efficiency [5].

With the rapid advancement of generative models in visual signal representation and processing, their impact has extended beyond academic research to international video coding standardization. Major standardization bodies, including the Joint Video Experts Team (JVET) and ISO/IEC MPEG, have actively explored neural network-based approaches to visual coding. A representative example is JPEG AI [6], a joint standardization effort between ISO/IEC JPEG and ITU-T, which adopts learning-based techniques for image compression while supporting downstream computer vision tasks. These efforts highlight neural video coding as a recognized and promising direction for future visual coding systems.

1.2 Problem Statement

Despite the rapid progress of neural video coding (NVC), rate control (RC) remains a critical component for its practical and commercial deployment. In real-time applications such as video conferencing and live streaming, bitrate is the primary physical resource constraint, and unstable rate control may lead to buffer overflow or underflow, resulting in latency, frame drops, inefficient bandwidth utilization, and degraded QoS. Therefore, a stable and reliable rate control mechanism is indispensable for real-time NVC systems.

Conventional rate control methods in hybrid video coding frameworks are typically derived from rate-distortion optimization (RDO) and rely on simplified and approximately stationary models that relate bitrate to the quantization parameter (QP) [7]. Although effective for traditional codecs, these assumptions no longer hold for modern coding systems with increasingly flexible and adaptive coding structures.



In NVC, end-to-end learned representations introduce highly nonlinear, frame-level, and content-dependent rate characteristics, causing the Rate-QP relationship to become non-stationary and difficult to model. As a result, directly applying conventional model-based rate control methods often leads to unstable quantization parameter selection and inaccurate bitrate estimation in practical deployment.

In summary, there is currently a lack of real-time rate control solutions that simultaneously achieve high accuracy, low computational complexity, and robust adaptability to modern NVC architectures. Addressing this challenge is essential for enabling the practical deployment of neural video coding systems.

1.3 Motivation

Throughout the development of NVC, encoding and decoding speed has long been one of the primary bottlenecks limiting practical deployment. The emergence of DCVC-RT [8], however, marks an unprecedented milestone. DCVC-RT successfully achieves a favorable balance between computational efficiency and compression performance. In most scenarios, it surpasses conventional codecs in both speed and coding efficiency, and even outperforms experimental software codecs such as ECM [9] that have yet to be officially released. This breakthrough effectively removes the long-standing perception that NVC is impractical due to excessive computational complexity, thereby enabling realistic prospects for commercial deployment.

In this context, efficient rate control has become the “last mile” toward large-scale deployment of NVC systems. Although DCVC-RT has demonstrated the coding potential of modern NVC architectures, research on rate control specifically tailored to such state-of-the-art (SOTA) NVC models remains limited. Most existing studies [10, 11, 12] focus

on earlier NVC architectures and rely on extensions of traditional mathematical models. Furthermore, due to the performance constraints of early NVC models, the experimental scenarios used in those studies often do not align with real-world rate control requirements—specifically real-time video conferencing or live streaming. Additionally, the rapid evolution of NVC architectures in recent years has made it difficult to establish unified and generalizable experimental frameworks.

As DCVC-RT is the first NVC architecture to achieve practical standards in both speed and comprehensive performance, investigating its Rate – QP relationship and designing a dedicated, lightweight rate control mechanism offers substantial academic and industrial value. Such research can also serve as a foundation for future developments in the field. Accordingly, this thesis focuses on DCVC-RT as a practically viable NVC model and aims to address the core rate control challenges that hinder the real-world deployment of neural video coding systems.

1.4 Primary Contributions

This thesis aims to address the aforementioned challenges by proposing an efficient real-time rate control framework for modern NVC architectures. The main contributions are summarized as follows:

1. Adaptive State-Space Modeling via Recursive Least Squares (RLS)

- We propose a lightweight adaptive state-space model designed to capture the nonlinear Rate–QP relationship in modern NVC architectures like DCVC-RT.
- To the best of our knowledge, this is the first study to introduce Recursive Least Squares (RLS) [13] into NVC rate control, enabling precise real-time

bitrate estimation.



2. Superior Balance of Accuracy and Efficiency

- While existing baselines are also capable of real-time processing, the proposed RLS-based system achieves higher bitrate accuracy and visual quality.
- The proposed framework introduces negligible computational overhead, thereby strictly maintaining the low-latency performance required for real-time video encoding.

3. Experimental Transparency and Reproducibility

- Unlike previous studies that often omit implementation details, this thesis explicitly documents critical experimental settings, including initialization strategies and parameter configurations.
- This transparency ensures reproducibility and provides a clear, reliable baseline for future research in NVC rate control.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces the fundamentals of video compression, including traditional hybrid coding frameworks and neural video compression (NVC). Chapter 3 reviews related work on state-of-the-art NVC architectures and existing rate control methods, and discusses their limitations. Chapter 4 presents the proposed adaptive state-space rate control framework based on recursive least squares (RLS). Chapter 5 evaluates the proposed method through experimental comparisons and parametric analysis. Finally, Chapter 6 concludes the thesis and outlines directions for future work.



Chapter 2

Fundamentals



2.1 Video Compression Basics

2.1.1 Information Theory – Entropy

To understand the theoretical limits of video compression, we introduce the fundamental concepts established by Claude Shannon [14]. The definitions in this section are primarily summarized from [15]. The core idea revolves around quantifying "information" and establishing a lower bound for the number of bits required to represent data.

Self-Information and Entropy

Shannon defined a quantity called *self-information* based on probability. Let A be an event with a probability $P(A)$. The self-information is defined as $i(A) = -\log_2 P(A)$. This aligns with the intuition that low-probability events (e.g., rare pixel values) contain high information, while high-probability events contain little.

For a source generating a sequence of independent and identically distributed (i.i.d) symbols X , the average self-information is known as the **Entropy** (H):

$$H(X) = -\sum P(X) \log_2 P(X) \quad (2.1)$$

Relevance to Compression

The significance of entropy lies in Shannon's Source Coding Theorem. It demonstrates that the best a lossless compression scheme can do is to encode the source with an

average number of bits equal to its entropy.



2.1.2 Redundancy

Raw video data require an extremely large amount of bandwidth. Efficient compression exploits inherent redundancies: spatial, temporal, and coding redundancy, along with perceptual redundancy in color space.

Spatial Redundancy

Spatial redundancy arises from the strong statistical correlation between neighboring pixels in natural images. As illustrated in Fig. 2.1, image content can generally be categorized into flat regions, edges, and textures. Flat regions (e.g., the smooth background) exhibit the highest redundancy with minimal pixel variance, whereas edges (e.g., facial boundaries) and textures (e.g., hair strands) contain high-frequency information.

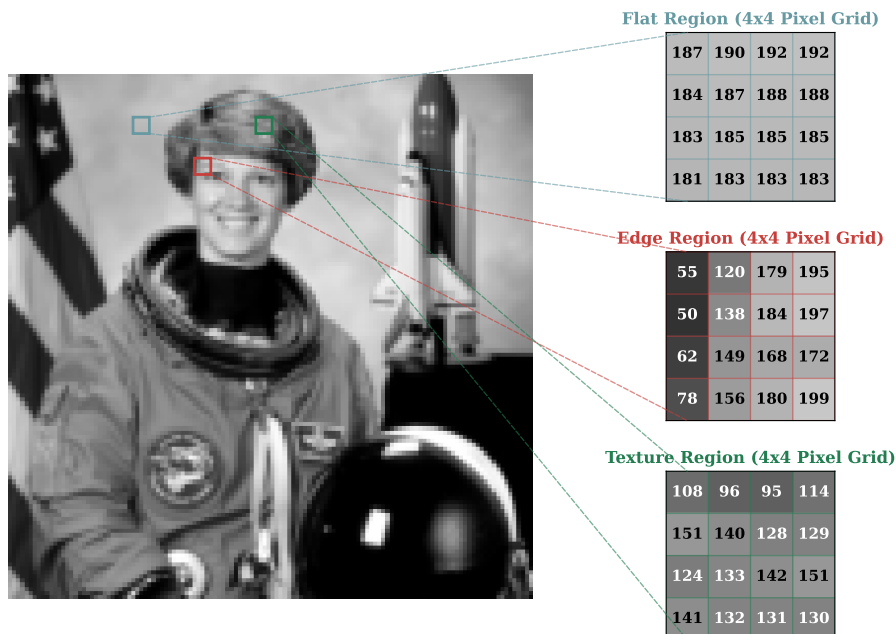


Figure 2.1: Spatial redundancy in the 'Astronaut' image: flat (background), edge (facial features), and texture (hair) regions.

To exploit the correlation between these neighboring pixels, **Differential Pulse-Code**

Modulation (DPCM) is historically employed. DPCM operates by predicting the current pixel x_n based on its reconstructed neighbor x_{n-1} and encoding the difference, known as the residual ($d_n = x_n - x_{n-1}$). As demonstrated in Fig. 2.2, while the original pixel intensities spans the full dynamic range (top), the prediction residuals form a **peaked Laplacian-like probability distribution** centered at zero (bottom). This energy compaction significantly reduces the entropy, thereby facilitating more efficient entropy coding.

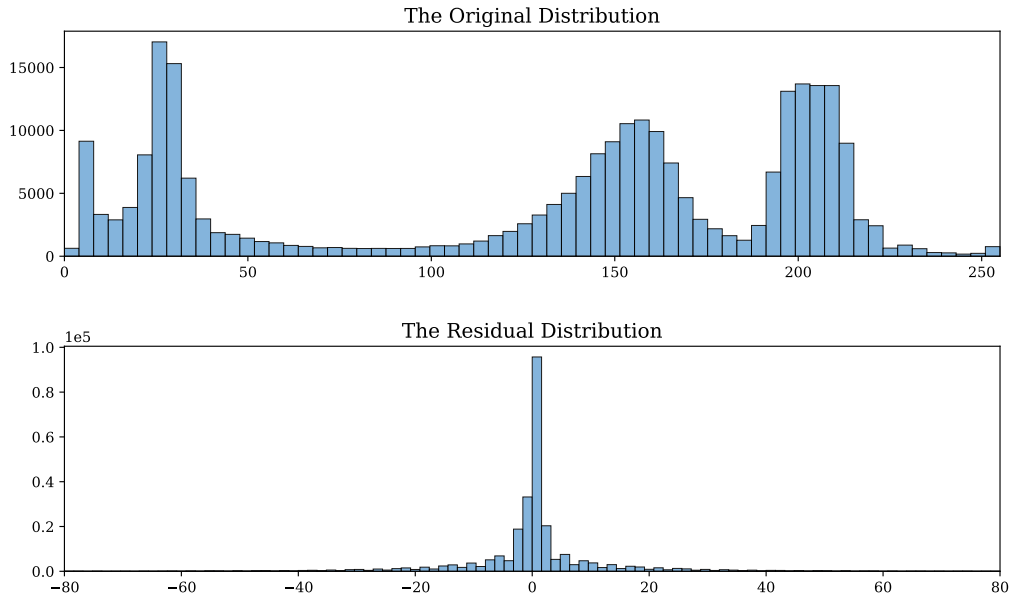


Figure 2.2: Statistical comparison: original pixel intensities distribution (top) vs. prediction residuals distribution (bottom).

Temporal Redundancy

Successive video frames, denoted as I_t and I_{t+1} , typically exhibit strong statistical correlation, a phenomenon known as **temporal redundancy**. To eliminate this redundancy, modern video coding standards employ **inter-frame prediction**, which consists of two core components:

1. **Motion Estimation (ME):** The encoder partitions the current frame into blocks and searches for the optimal matching block in the reference frame. The spatial

displacement is encoded as a **Motion Vector (MV)**.

2. **Motion Compensation (MC):** The reference frame is warped using the derived MVs to synthesize a prediction of the current frame.

The coding efficiency is largely determined by the **residual**, defined as the difference between the actual current frame and the prediction. Fig. 2.3 visualizes this process using a sample from the Vimeo-90k dataset [16]. Due to camera motion, the direct difference between frames results in significant high-frequency energy along object boundaries. In contrast, applying motion compensation effectively aligns the temporal features, reducing the residual to a sparse, noise-like signal.

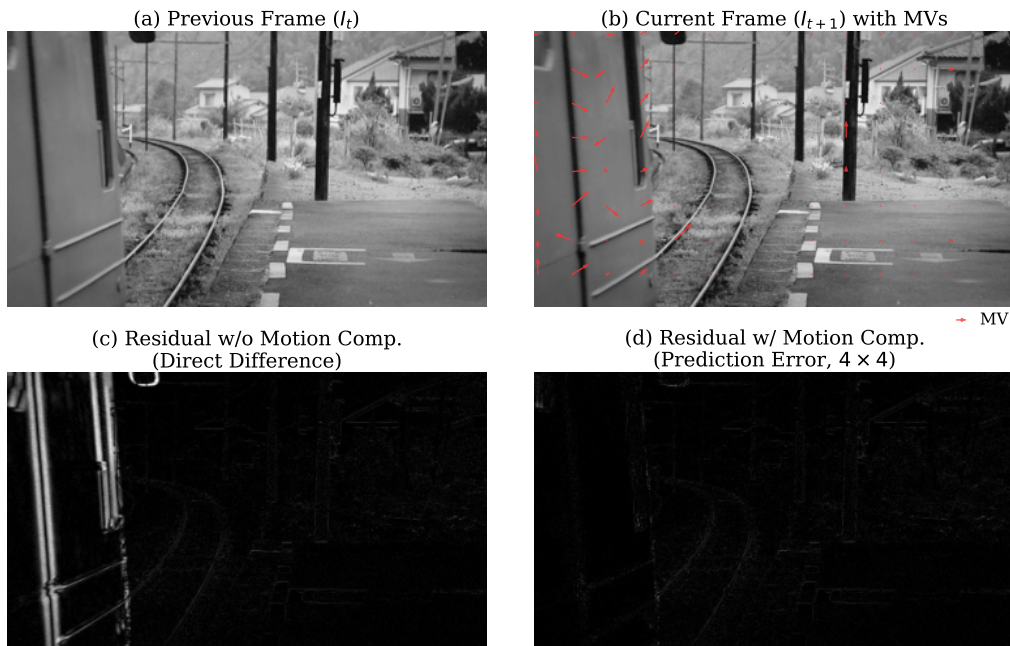
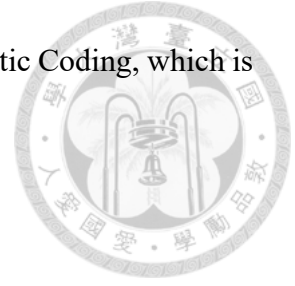


Figure 2.3: Temporal redundancy reduction: motion estimation (top) and residual comparison (bottom).

Coding Redundancy

If we use a fixed number of bits for every symbol, we waste space. **Variable-Length Coding (VLC)** solves this by assigning short codes to frequent symbols and long codes to rare ones. Classical methods like Huffman coding [17] use this principle to approach

the theoretical entropy limit. Modern standards typically use Arithmetic Coding, which is discussed later in Section 2.2.5.



Color Space Representation

Videos are captured in RGB, but the human eye is more sensitive to brightness than color. To save data, we convert RGB into the YUV color space. The Luminance (Y) keeps the details, while the Chrominance (U, V) is often reduced in resolution (e.g., 4:2:0 subsampling). This reduces the file size with almost no visible loss in quality.

2.1.3 The Rate-Distortion (R-D) Trade-off

While entropy sets the theoretical limit for lossless compression, practical video coding relies on **lossy compression** to satisfy bandwidth constraints. This necessitates a fundamental trade-off: minimizing Bitrate (R) while minimizing Distortion (D).

Quantization and Reconstruction

In both hybrid frameworks and Neural Video Coding (NVC), **Quantization** serves as the decisive mechanism to govern this trade-off. It maps high-precision transform coefficients (or latent features) to a finite set of discrete levels, thereby reducing information entropy.

Figure 2.4 illustrates this concept using a block-based quantization matrix, common in traditional standards. The process typically involves division by a step size (or matrix element), followed by a rounding operation. The figure demonstrates the impact of quantization granularity:

- **High Quality Path (Top):** Using fine quantization steps preserves details. The

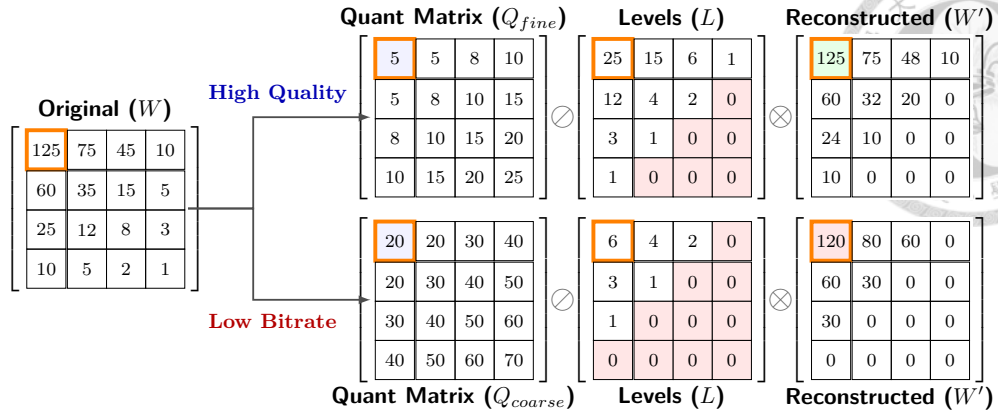


Figure 2.4: The mechanics of quantization. Coarse quantization (bottom path) reduces bitrate at the cost of reconstruction fidelity.

reconstruction is nearly lossless (e.g., the value is restored as 125), but the result retains high entropy.

- **Low Bitrate Path (Bottom):** Using coarse quantization forces many small values to zero. While this sparsity significantly reduces the bitrate, it introduces irreversible error (e.g., the value becomes 120).

Lagrangian Rate-Distortion Optimization

The relationship between rate and distortion is empirically convex, as shown in the R-D curves in Fig.2.5. Initially, a small increase in bitrate yields significant quality gains, but the returns eventually diminish.

To automate the selection of the optimal operating point along these curves, modern encoders employ **Rate-Distortion Optimization (RDO)** using the method of Lagrange multipliers. The system seeks to minimize a joint cost function J :

$$\min J = D + \lambda R \quad (2.2)$$

Here, the Lagrange multiplier λ acts as a trade-off parameter. A larger λ imposes a heavy penalty on rate, pushing the encoder towards the "Low Bitrate" path, whereas a smaller λ prioritizes reconstruction fidelity.

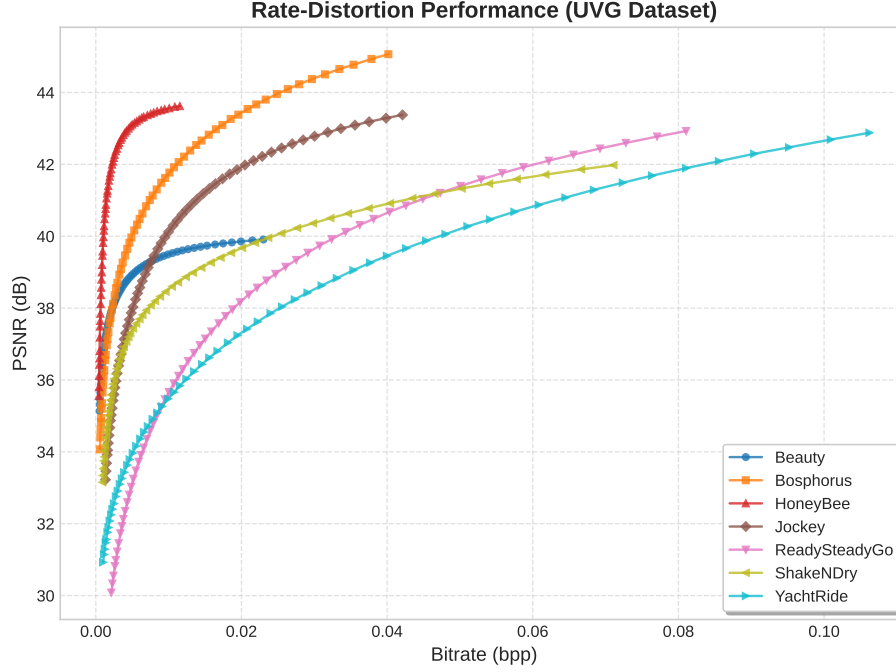


Figure 2.5: Rate-Distortion (R-D) performance curves evaluated on the UVG dataset, illustrating the trade-off between quality (PSNR) and bitrate (bpp).

2.2 From Traditional Standards to Neural Video Coding

2.2.1 The Generalized Hybrid Coding Pipeline

Modern video compression standards, such as H.264/AVC [2] and H.265/HEVC [3], have dominated the industry for decades. These standards rely on the classic *hybrid coding architecture*, which combines predictive coding (to exploit temporal redundancy) and transform coding (to exploit spatial redundancy). Interestingly, the emergence of Neural Video Coding (NVC) does not necessarily discard this proven architecture. Instead, early pioneering works, such as the Deep Video Compression (DVC) framework by Lu et al. [18], propose an end-to-end learning framework that maintains a one-to-one correspondence with traditional standards.

Fig. 2.6 illustrates this architectural evolution. On the left, the traditional framework consists of hand-crafted modules: block-based motion estimation, integer transform derived from discrete cosine transform (DCT), quantization, and entropy coding. Although each module is well-designed, they are optimized separately, which may not lead to the global optimum for the entire system.

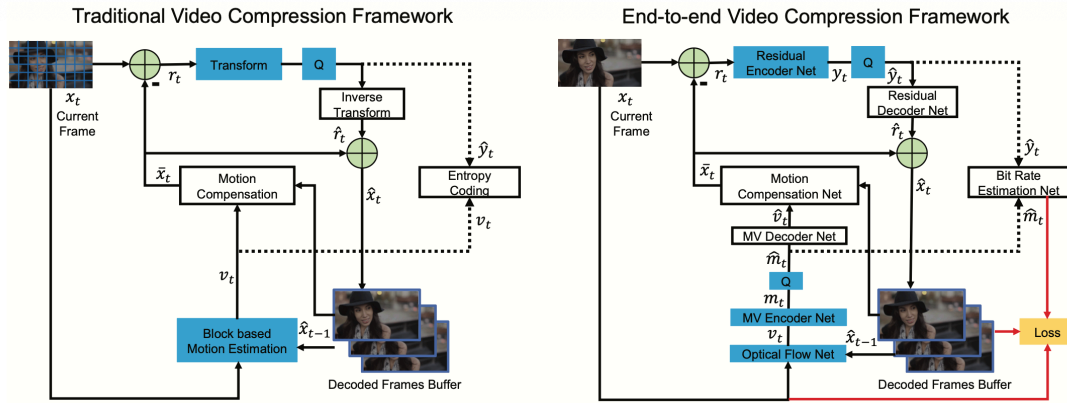


Figure 2.6: Comparison between the traditional predictive coding architecture (left) and the end-to-end Deep Video Compression (DVC) framework (right) [18].

On the right side of Fig. 2.6, the DVC framework replaces these key components with neural networks:

- **Motion Estimation & Compensation:** The traditional block-based search is replaced by an Optical Flow Net and a Motion Compensation Net to perform pixel-wise prediction.
- **Transform & Inverse Transform:** The linear transform is replaced by non-linear Residual Encoder and Decoder networks, which utilize the powerful representation ability of Convolutional Neural Networks (CNNs) [19].
- **Entropy Coding:** The hand-designed context models are replaced by CNN-based Bit Rate Estimation Nets to estimate the probability distribution of latent representations.

The most significant advantage of this approach is the capability for *joint optimization*. Unlike traditional codecs where modules are tuned individually, all components in the NVC framework—including motion estimation, compression, and residual coding—are implemented as neural networks and optimized simultaneously through a single loss function. This objective considers the trade-off between the number of compression bits and the reconstruction quality, allowing the modules to collaborate effectively to improve overall coding efficiency.

2.2.2 Temporal Prediction: Motion Vectors to Optical Flow

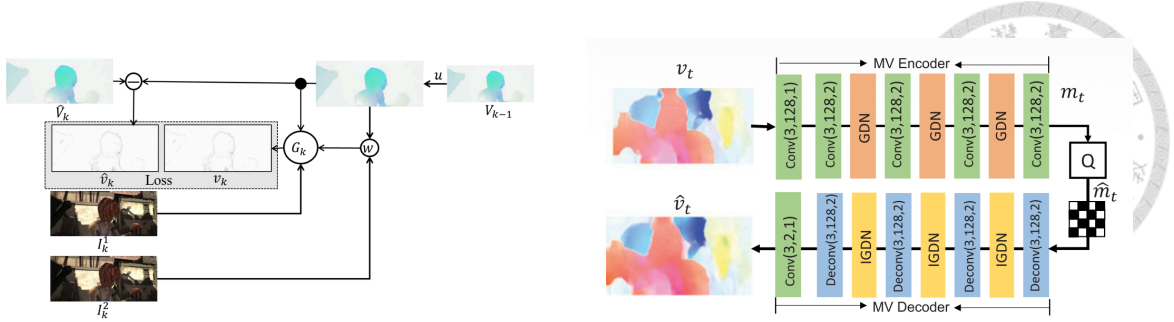
Traditional codecs split the image into blocks and assume objects move rigidly. This often fails for complex motions (like deformation). NVC solves this by using **Optical Flow**, which estimates motion for every single pixel.

Pyramid-based Motion Estimation

DVC uses a network called SPyNet [20] (see Fig. 2.7(a)). It uses a "pyramid" structure to handle both large and small movements efficiently. Through joint training, the network learns to generate motion fields that are accurate but also easy to compress.

Motion Compensation and Compression

After warping the frame with optical flow, a **Motion Compensation Network** refines the result to remove artifacts. Since pixel-wise optical flow is very large, it must be compressed. DVC uses an **MV Encoder-Decoder** (Fig. 2.7(b)) to compress the motion field into a small latent representation.



(a) Optical Flow Estimation (SPyNet) [20].

(b) MV Encoder and Decoder Network [18].

Figure 2.7: The motion processing pipeline in DVC.

2.2.3 Spatial Transformation: Linear to Non-linear

Traditional standards use the Discrete Cosine Transform (DCT) [21]. DCT is linear and processes images in fixed 8×8 blocks. This can cause "blocking artifacts" and cannot capture complex structures that cross block boundaries.

NVC replaces this with **Learned Non-linear Transforms** (CNNs with GDN layers [22]). As shown in Fig. 2.8, the learned features (c) are not limited to blocks. They automatically focus on important structures like object boundaries. This allows the system to allocate bits more intelligently based on visual importance.

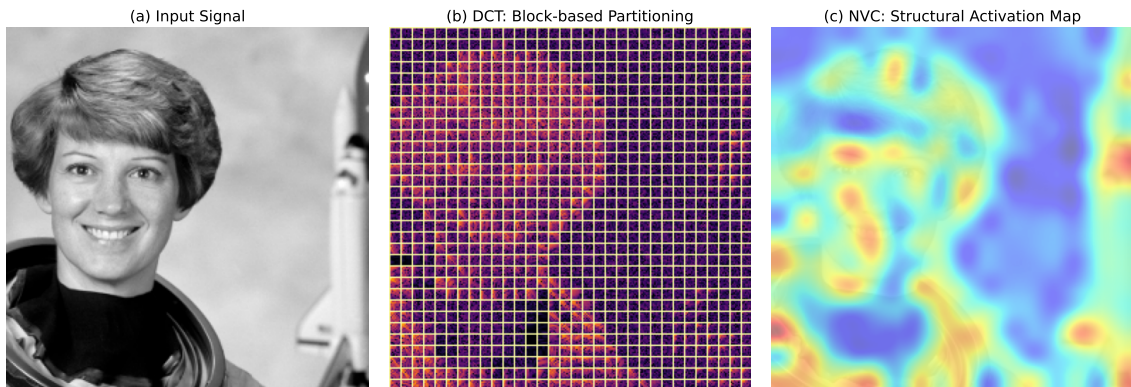


Figure 2.8: Visual comparison of spatial energy distribution. The NVC model [23] extracts semantic features (c) using **CompressAI** [24], unlike the rigid blocks in traditional DCT (b).

2.2.4 Differentiable Quantization and Optimization

While quantization is essential for compression, it poses a difficult problem for deep learning: the standard rounding operation, $Q(y) = \text{round}(y)$, is non-differentiable. Because its gradient is zero almost everywhere, it effectively cuts off the gradient flow needed for back-propagation.

Training Proxy: Uniform Noise Approximation

To make the system differentiable, NVC frameworks replace the rigid rounding operation with **additive uniform noise** during the *training phase*, a method proposed by Ballé et al. [25].

As shown in Figure 2.9 (Right), we approximate the discrete probability mass function of the quantized symbol $\hat{y}$ using the continuous density function of a noisy symbol $\tilde{y}$:

$$\tilde{y} = y + \eta, \quad \text{where } \eta \sim \mathcal{U}(-0.5, 0.5) \quad (2.3)$$

This **noise approximation** effectively simulates the error introduced by quantization ($\hat{y} - y$) while ensuring the gradients remain valid for optimization. Once training is complete, the model switches back to standard rounding ($\hat{y} = \text{round}(y)$) for the actual inference.

End-to-End Loss and Rate Estimation

Using this differentiable proxy allows us to train the entire compression system jointly. The Rate-Distortion Optimization (RDO) is directly integrated into the training loss:

$$\mathcal{L} = D(x, \hat{x}) + \lambda R(\tilde{y}) \quad (2.4)$$

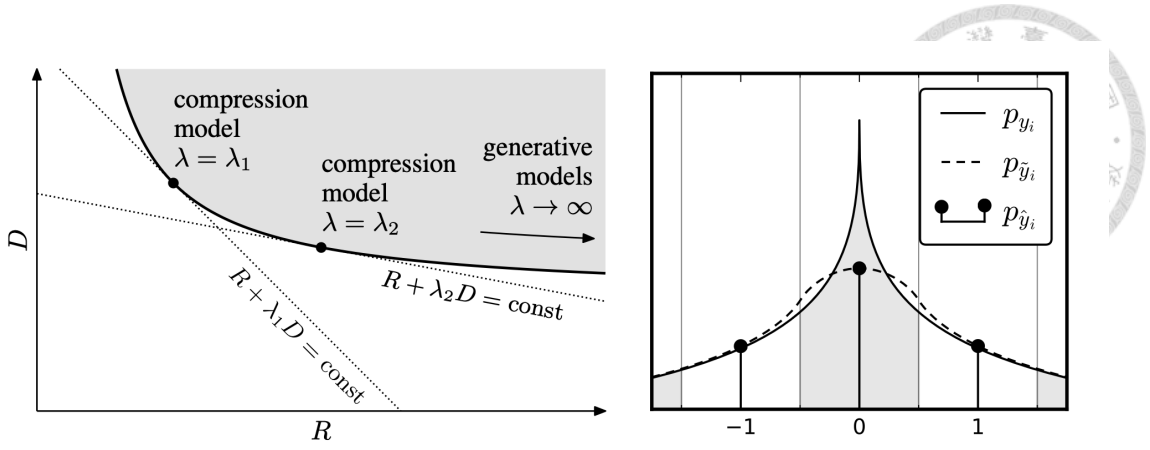


Figure 2.9: Quantization strategies in NVC. Left: The optimization landscape defined by λ . Right: To resolve the zero-gradient problem of hard quantization (dots), uniform noise (dashed line) is added during training as a differentiable proxy [25].

While calculating distortion D (e.g., MSE, MSSSIM) is straightforward, the rate term $R(\tilde{y})$ presents a challenge because counting the actual bits from an arithmetic coder is a non-differentiable process.

To address this, we rely on Shannon’s source coding theorem[14], which states that the lower bound of the bit rate is the **Cross Entropy** between the marginal distribution of the latents and the learned probability model. Therefore, instead of counting bits, the loss function estimates the rate R as the **Negative Log-Likelihood (NLL)** of the noisy symbols:

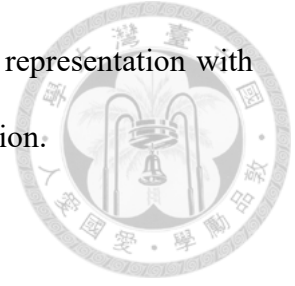
$$R(\tilde{y}) = \mathbb{E}_{\tilde{y} \sim p_{\tilde{y}}}[-\log_2 p_{\theta}(\tilde{y})] \quad (2.5)$$

where $p_{\theta}(\cdot)$ is the probability distribution predicted by the Entropy Model (see Section 2.2.5).

In our implementation, as the latent variables are convolved with a unit uniform distribution (due to the quantization noise), the probability of a symbol $\tilde{y}$ is calculated by integrating the learned distribution density (e.g., Laplace or Gaussian) over the corresponding quantization bin:

$$p_{\theta}(\tilde{y}) = \text{CDF}_{\theta}(\tilde{y} + 0.5) - \text{CDF}_{\theta}(\tilde{y} - 0.5) \quad (2.6)$$

Minimizing this NLL loss encourages the network to learn a latent representation with lower entropy, effectively reducing the bit rate required for transmission.



2.2.5 Entropy Coding: Context Models to Learned Priors

After quantization, the resulting discrete symbols must be compressed into a compact bitstream. While traditional standards employ fixed or adaptive lookup tables, NVC relies on Arithmetic Coding driven by learned probability distributions.

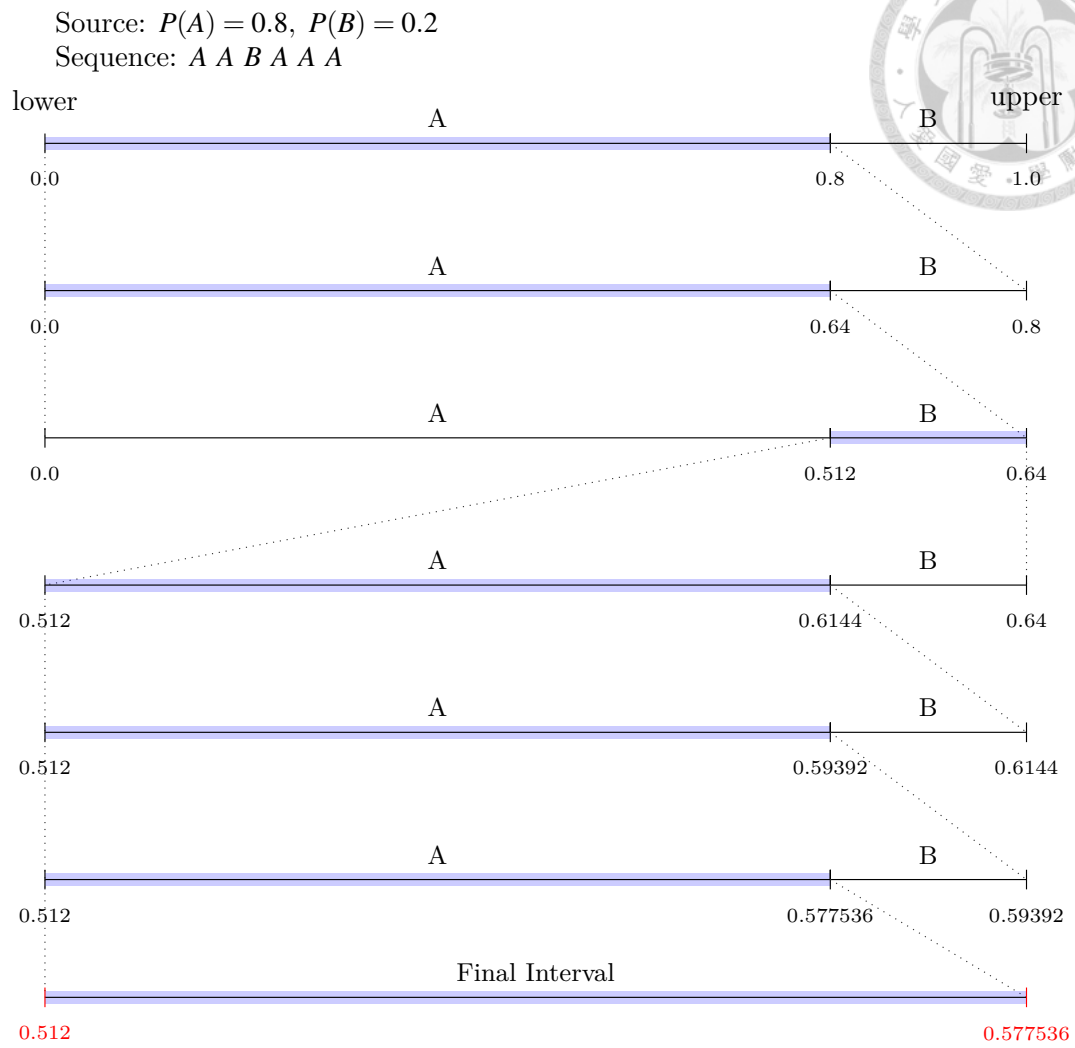
From Huffman to Arithmetic Coding

Traditional Huffman coding maps symbols to integer-length codewords. While optimal for symbol-by-symbol coding, this integer constraint becomes a significant bottleneck when handling symbols with very high probabilities (e.g., if $P(A) = 0.9$, the ideal code length is $-\log_2(0.9) \approx 0.15$ bits, but Huffman must assign at least 1 bit).

Arithmetic Coding (AC) overcomes this limitation by encoding a sequence of symbols into a single floating-point number, effectively allowing for the allocation of fractional bits. As illustrated in Figure 2.10, the probability interval $[0, 1)$ is recursively subdivided. A high-probability symbol retains a larger sub-interval, requiring fewer bits to resolve the final coordinate. This property allows AC to approach the theoretical **Entropy** limit (H) much more closely than Huffman coding.

Context Modeling and CABAC

The efficiency of Arithmetic Coding is strictly bounded by how accurately the estimated probability $P(\hat{y})$ matches the true data distribution. In H.264/AVC and H.265/HEVC, this is handled by **CABAC (Context-based Adaptive Binary Arithmetic Cod-**



Final Interval: $[0.512, 0.577536)$
 $0.512 \leq 17 \cdot 2^{-5} < 18 \cdot 2^{-5} \leq 0.577536$
 Output Code(convert 17 to binary form): 10001 (5 bits)
 Efficiency: $5/6 \approx 0.83$ bits/symbol

Figure 2.10: Illustration of Binary Arithmetic Coding.

ing) [26]. CABAC utilizes hand-crafted context models that update on-the-fly based on previously coded symbols (neighbors). NVC adopts a similar philosophy but replaces these manual context updates with neural networks.

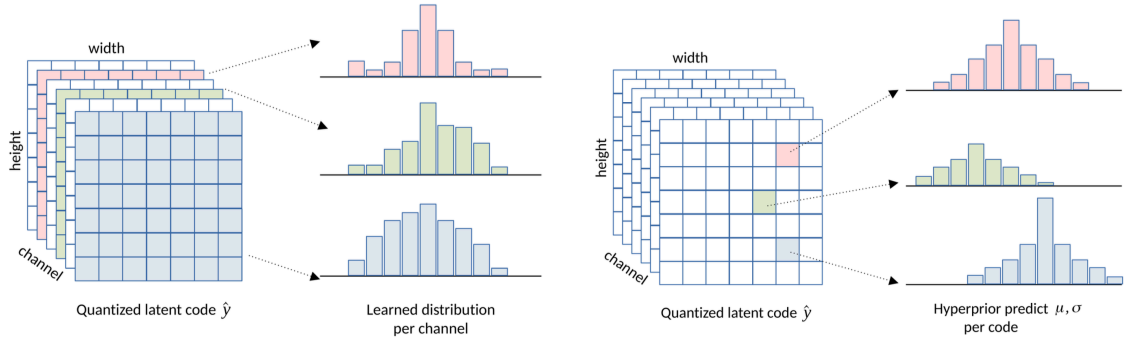


Learned Entropy Models

To provide precise probability estimates for the arithmetic coder, NVC employs CNN-based "Entropy Models".

Early approaches used a **Factorized Prior** (Figure 2.11(a)), assuming that each channel follows a fixed, independent distribution. However, this fails to capture the spatial variation of natural images—complex textures and smooth backgrounds often coexist in the same channel.

To address this, modern frameworks (e.g., DVC [18]) adopt the **Hyperprior** model (Figure 2.11(b)). This approach models the latent $\hat{y}$ as a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ where the parameters are spatially adaptive.



(a) Factorized Prior: Independent distributions. (b) Hyperprior: Spatially adaptive distributions.
Figure 2.11: Conceptual comparison of Entropy Models [27].

Figure 2.12 details the operational flow of the Hyperprior architecture. The network utilizes a hierarchical structure:

1. A **Hyper-Encoder** (h_a) summarizes the spatial information of the latent $\hat{y}$ into a side-information variable $\hat{z}$.
2. This $\hat{z}$ is transmitted first (as side info).
3. A **Hyper-Decoder** (h_s) uses $\hat{z}$ to estimate the mean and standard deviation scales

(μ, σ) of the Gaussian distribution for each pixel in $\hat{y}$.

By conditioning the probability model on these parameters, the arithmetic coder can allocate more bits to complex regions (high σ) and fewer to smooth regions, significantly improving the rate-distortion performance.

Crucially, the parameters μ and σ predicted by the Hyper-Decoder are precisely what define the probability distribution p_θ used in the rate estimation loss (Eq. 2.5) discussed in Section 2.2.4. This end-to-end linkage ensures that the network minimizes the bitrate by learning to predict distributions that tightly match the actual statistics of the latent codes.

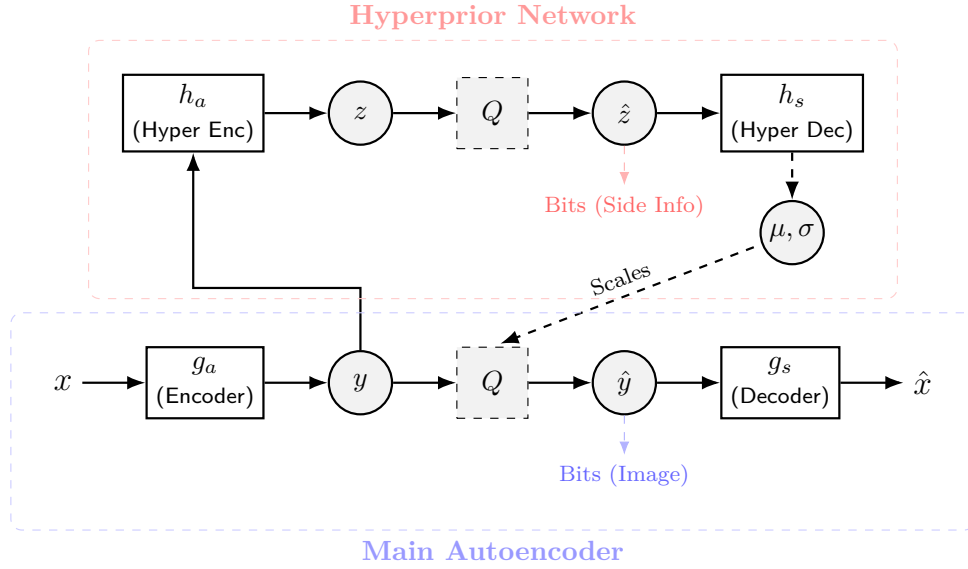


Figure 2.12: Operational diagram of the Hyperprior architecture. The input y is analyzed by the hyper-branch (top) to generate compressed side-information $\hat{z}$. The reconstructed $\hat{z}$ is then decoded to predict the mean and scales (μ, σ) , which model the statistical distribution of $\hat{y}$ for efficient arithmetic coding.

Chapter 3

Literature Review

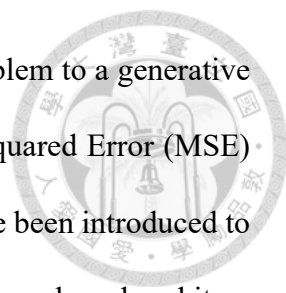


3.1 State-of-the-Art Neural Video Codecs

3.1.1 Developmental Progression

The foundation of Neural Video Coding (NVC) is fundamentally built upon advancements in Learned Image Compression (LIC). Ballé et al. [25] pioneered the use of Variational Autoencoders (VAEs) in this domain. To effectively capture spatial dependencies and match the statistics of natural images, Ballé further introduced the hyperprior model [23, 28] and Generalized Divisive Normalization (GDN) [22]. Unlike simple linear operations, GDN serves as a parametric non-linear transformation that effectively Gaussianizes the data density. By minimizing the mutual information between transformed components, GDN significantly enhances the efficiency of subsequent entropy coding, remaining a cornerstone in high-performance coding models [29].

Building on the success of LIC, Lu et al. proposed DVC (Deep Video Compression) [18, 30], the first end-to-end framework to successfully map the traditional hybrid coding pipeline—comprising motion estimation, motion compensation, and residual coding—onto neural networks. Early NVC was dominated by this "Residual Coding" paradigm, utilizing Optical Flow techniques to capture temporal redundancy. To address occlusions and large displacements, Agustsson et al. introduced Scale-space flow [20, 31, 32], incorporating a blurring field to improve prediction robustness. This design has become foundational for mainstream architectures [33, 34] and subsequent video analysis tasks [24, 35].



Recently, NVC has begun shifting from a signal processing problem to a generative modeling problem [36]. To surpass the perceptual limits of Mean Squared Error (MSE) optimization, Generative Adversarial Networks (GANs) [37, 38] have been introduced to reconstruct realistic textures at low bitrates. Concurrently, Transformer-based architectures [5, 39, 40, 41, 42] are being explored to leverage attention mechanisms for global spatiotemporal feature interaction. However, finding the optimal balance between computational complexity and compression efficiency remains an ongoing challenge compared to mature CNN-based approaches.

Despite these advancements, a critical limitation of early methods was their reliance on pixel-domain residual coding. Subtracting predicted frames from current frames often leads to high-frequency information loss, particularly in high-texture or complex motion scenes. This bottleneck spurred the development of Conditional Coding—the core philosophy behind the DCVC series—which shifts operations into the feature domain to better preserve spatiotemporal correlations.

3.1.2 Conditional Coding and DCVC Architecture

From Residual to Conditional Coding

Traditional codecs typically adopt a “residual coding” strategy, encoding the difference $r_t = x_t - \tilde{x}_t$. However, this linear subtraction often yields high-frequency, hard-to-compress signals and limits the modeling of non-linear spatiotemporal correlations. From an information-theoretic perspective, residual coding is merely a restricted special case of conditional coding. The entropy of residual coding is theoretically bounded by the entropy of conditional coding:



$$H(x_t - \hat{x}_t) \geq H(x_t | \hat{x}_t) \quad (3.1)$$

This inequality implies that relying solely on subtraction fails to fully exploit temporal correlations [43]. To break this theoretical bottleneck, Li et al. proposed the **Deep Contextual Video Compression (DCVC)** [44] framework. Instead of compressing pixel-domain differences, DCVC reformulates reconstruction as a conditional generation process:

$$\hat{x}_t = f_{dec}(\lfloor f_{enc}(x_t | \bar{x}_t) \rfloor | \bar{x}_t) \quad (3.2)$$

where $\bar{x}_t$ is a high-dimensional context learned in the feature domain. As illustrated in Fig. 3.1, unlike standard predicted frames, this feature-rich context carries semantic information and texture patterns, enabling the entropy model to capture more accurate temporal priors.

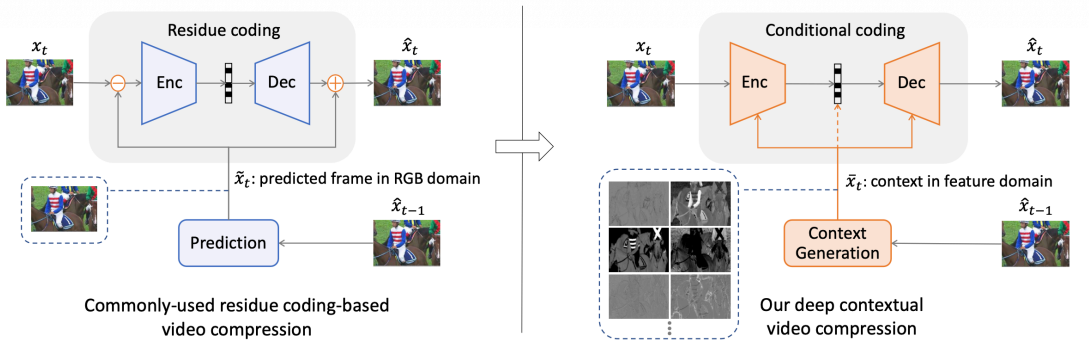


Figure 3.1: The paradigm shift from a residue coding-based framework to a conditional coding-based framework [44].

Temporal Context Mining (TCM)

While DCVC established the effectiveness of conditional coding, its initial implementation derived contexts from the previously decoded frame $\hat{x}_{t-1}$, which loses information during the projection back to the pixel domain. To extract more accurate temporal

information, DCVC-TCM [45] introduces the **Temporal Context Mining (TCM)** module (Fig. 3.2).



- **Feature Propagation:** Instead of using reconstructed frames, TCM propagates feature maps F_{t-1} directly. These features retain high-dimensional information that has not been degraded by RGB projection, ensuring better temporal coherence.
- **Multi-scale Context Mining:** Recognizing that motion and texture exhibit spatiotemporal non-uniformity, TCM employs a hierarchical structure to generate multi-scale contexts. The module performs feature warping and refinement across different resolutions to capture both large-scale motion and fine-grained details simultaneously.
- **Temporal Context Re-filling:** The mined contexts are injected into the encoder, decoder, and entropy model, ensuring that temporal information is utilized at every stage of the compression pipeline.

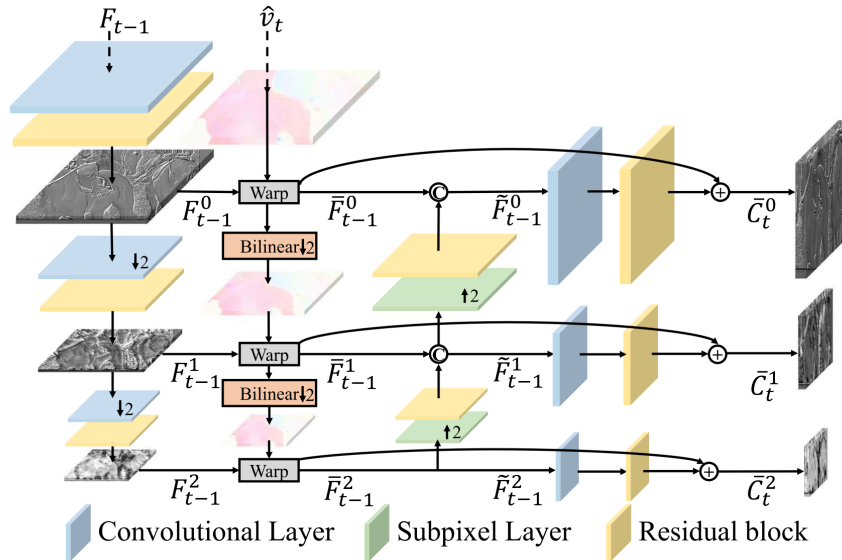


Figure 3.2: Architecture of the temporal context mining (TCM) module [45].



3.1.3 Single Model, Variable R-D Performance

In early NVC, a separate model had to be trained for each target bitrate by optimizing with a specific Lagrange multiplier (λ). This requirement led to high storage costs. Although transitional approaches such as internal feature modulation [46] and slimmable decoders [47] attempted to add adaptability, they failed to achieve continuous and smooth quality range within a single framework.

Adaptability via Quantization Scaling

Li et al. addressed this in **DCVC-HEM** [48] by introducing a quantization scaler (q_s) as a conditional input to the network. As shown in Fig. 3.3, the q_s interacts with the latent features in both the encoder and decoder. This mechanism allows the model to dynamically adjust the quantization step size and modify the feature distribution during inference. Consequently, a single trained model can support a wide range of bitrates without the need for retraining or storing multiple weights.

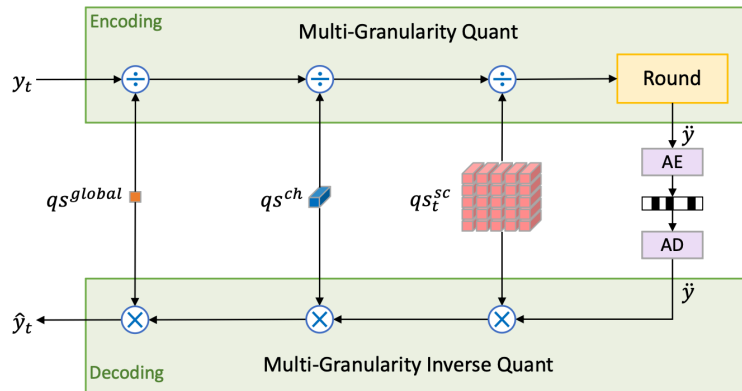


Figure 3.3: Multi-granularity quantization and the corresponding inverse quantization [48].

Quadtree Partition-Based Entropy Coding

To further enhance compression efficiency, recent research has focused on sophisti-

cated probabilistic modeling. While methods like hierarchical predictive learning [49] or multi-mode ensembles [50] have shown promise, **DCVC-DC** [51] pushes this boundary further by refining the probability estimation through a "Diverse Contexts" architecture featuring a **Quadtree Partition** strategy.

As illustrated in Fig. 3.4, the latent representation $\hat{y}_t$ is split into channel groups and processed in four steps (Step 0 to Step 3).

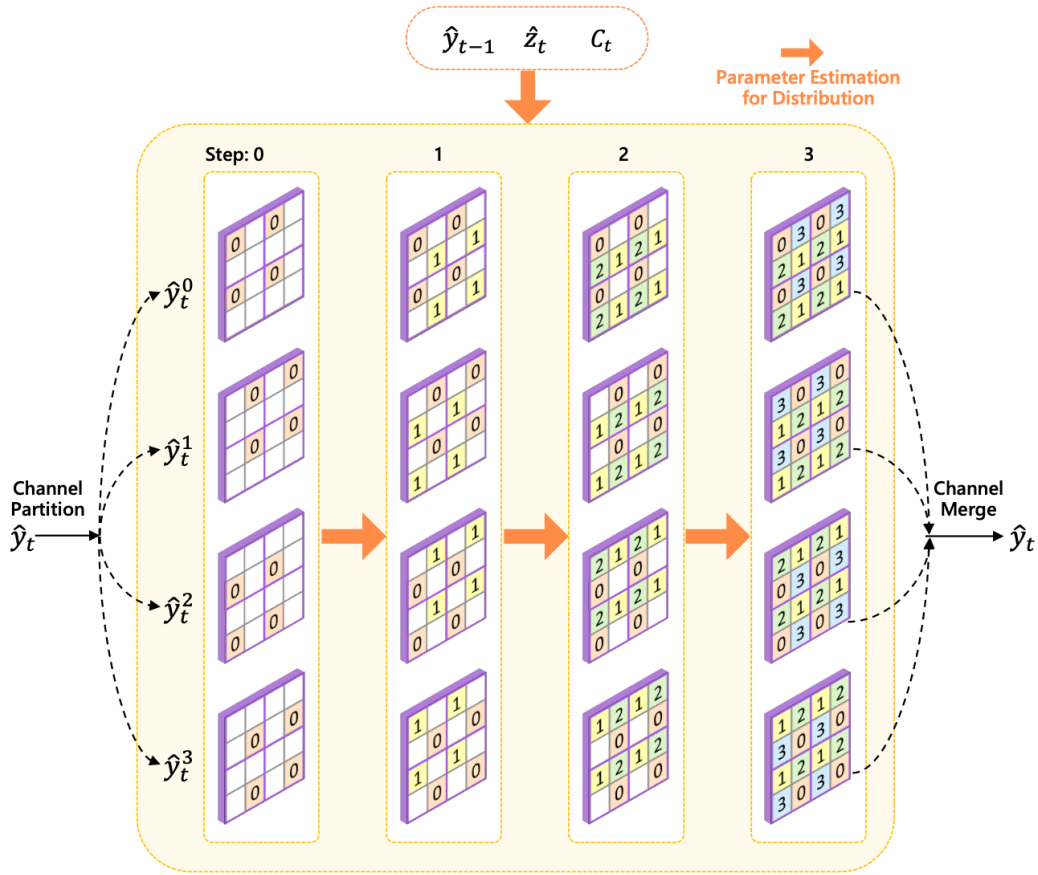


Figure 3.4: Entropy coding with quadtree partition. It fully mines the correlation from both spatial and channel dimensions [51].

1. **Step 0:** The anchor channels are coded independently without spatial neighbors.
2. **Steps 1-3:** Subsequent steps utilize the previously coded channels as contexts, employing 4, 4, and 8 neighbors, respectively.

This design allows the model to exploit both spatial neighbors (via the checkerboard-like pattern in the quadtree) and cross-channel correlations simultaneously. This approach significantly refines probability estimation compared to simple hyperpriors, setting a new standard for NVC entropy coding.

Solving the GOP Constraint: Hierarchical Quality Structure

To address severe error propagation in large Group of Pictures (GOP) [52], DCVC-DC introduces a **Hierarchical Quality Structure** inspired by H.266/VVC [4]. This strategy imposes a periodic quality pattern by assigning variable weights (w_t) to the distortion loss:

$$L = \frac{1}{T} \sum_t^T (w_t \cdot \lambda \cdot D + R) \quad (3.3)$$

Key frames receive larger weights (e.g., 1.2) to serve as high-quality references, while dependent frames are assigned smaller weights (e.g., 0.5). As shown in Fig. 3.5, this results in a "zig-zag" PSNR fluctuation similar to H.266. These periodic anchors effectively reset accumulated errors, stabilizing performance across long sequences and preventing the quality collapse common in earlier neural codecs.

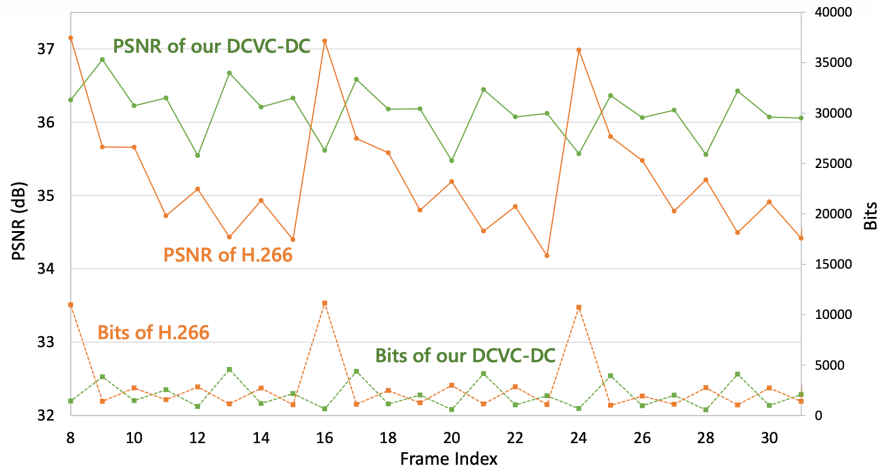


Figure 3.5: Hierarchical quality structure comparison [51].

3.1.4 Wider Quality Range and Real-Time Adaptation

Feature Modulation for Wide Range Coverage

While DCVC-DC enabled variable bitrates, performance often degraded at quality extremes. **DCVC-FM** [53] addresses this via a "Feature Modulation" mechanism, applying learnable scaling factors in the feature domain to dynamically adjust the dynamic range based on target quality (Fig. 3.6). As shown in Fig. 3.7, this yields a wide 11.3 dB quality range, outperforming DCVC-DC and reducing bitrate by 30.2% over VTM-17.0. Crucially, this refined modulation ensures stability in long-sequence "Single Intra-frame" coding, preventing the quality collapse often seen in earlier NVCs and offering robust support for potentially infinite video streams.

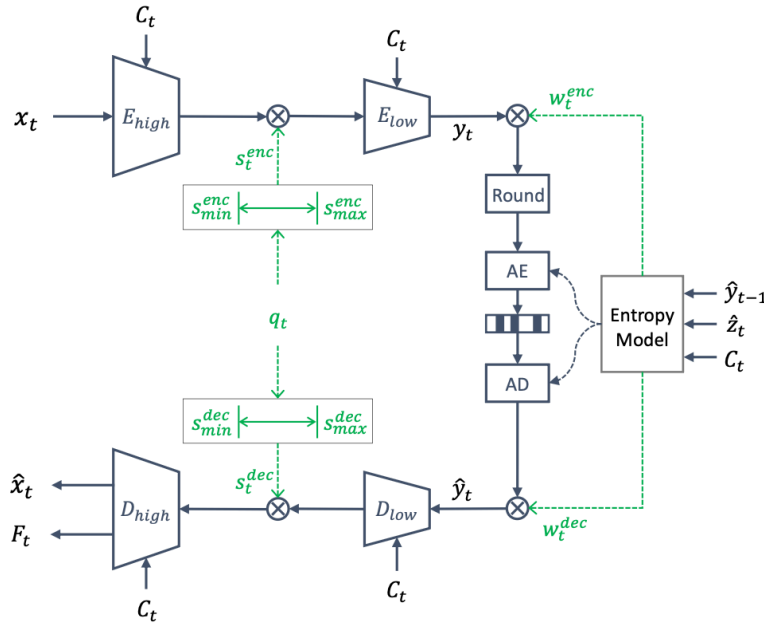


Figure 3.6: The Feature Modulation mechanism in DCVC-FM [53].

High-Throughput Neural Video Coding: DCVC-RT

While NVC offers superior compression, computational complexity has historically impeded real-time deployment. Previous acceleration efforts primarily focused on reduc-

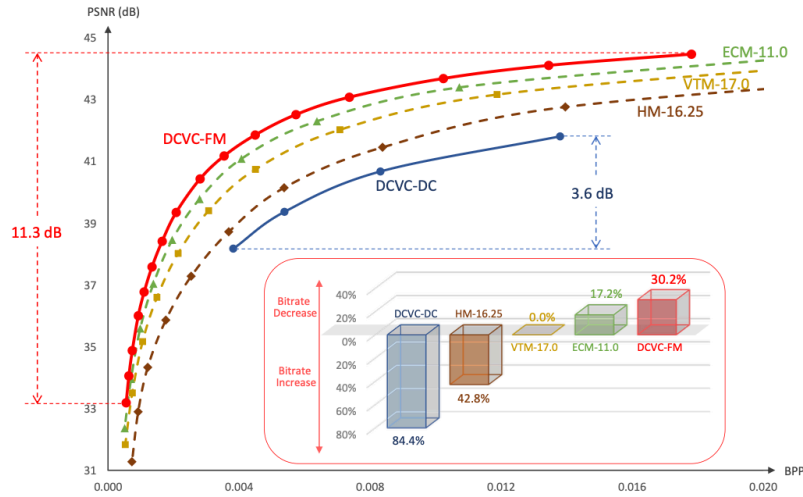


Figure 3.7: Rate-distortion performance comparison [53].

ing MACs [54, 55, 56], yet often **hit a speed limit** due to high ”Operational Complexity”—specifically, the heavy overhead from memory access (I/O) and frequent function calls. To break this bottleneck, **DCVC-RT** [8] rethinks the coding paradigm by targeting system-level efficiency (Fig. 3.8):

- **Implicit Temporal Modeling:** Replaces expensive optical flow with a lightweight feature extractor to model correlations directly in the latent space (Fig. 3.8b), drastically reducing module calls.
- **Single-Scale Latent Representation:** Processes latents at a single low resolution (1/8) instead of a progressive pyramid. This minimizes high-resolution memory I/O while maintaining a sufficient receptive field.
- **Practicality Enhancements:** Incorporates model integerization (Int16) and module-bank-based rate control to ensure cross-device consistency and adaptability.

By addressing these operational bottlenecks, DCVC-RT achieves 1080p/30fps real-time performance on consumer GPUs with rate-distortion performance comparable to the sophisticated DCVC-FM.

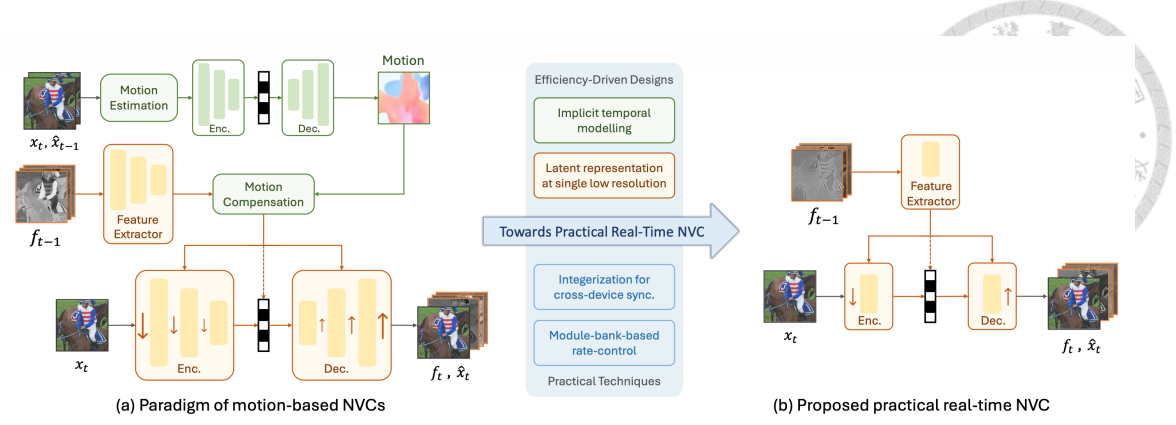


Figure 3.8: Comparison between (a) the traditional motion-based NVC paradigm and (b) the proposed DCVC-RT architecture. The shift to implicit modeling and single-scale latents significantly reduces operational complexity [8].

3.1.5 Summary

This section traced the evolutionary trajectory of Neural Video Coding (NVC), highlighting the decisive paradigm shift from pixel-domain residual coding to feature-domain conditional coding. Through the progressive development of the DCVC family—ranging from refined temporal context mining (DCVC-TCM) to versatile adaptability and real-time processing (DCVC-HEM, DCVC-RT)—NVC has successfully overcome early bottlenecks in accuracy and computational complexity. These advancements have elevated NVC from a theoretical exploration to a robust technology that consistently outperforms traditional standards like H.266/VVC in rate-distortion performance.

With the fundamental architecture now matured and superior compression efficiency established, the research focus naturally extends to system-level manageability. To bridge the gap between high-performance compression and practical video streaming, precise bitrate regulation becomes critical. This context underscores the value and necessity of developing dedicated rate control mechanisms for neural video codecs, which serves as the primary motivation for the subsequent discussion.

3.2 Rate Control Algorithms and Limitations



3.2.1 Rate Control in Traditional Codecs

Rate control regulates the encoded bitstream to satisfy bandwidth or buffer constraints while minimizing coding distortion. It is commonly posed as a constrained optimization problem: minimize distortion D subject to a target rate R_c .

Limitations of Prior Models ($R-Q$ and ρ -Domain)

Early standards (e.g., MPEG-2 and H.264/AVC) largely relied on parametric $R-Q$ models, including the quadratic model [57], implicitly treating the Quantization Parameter (QP) as the dominant (or even sole) control knob for bitrate. This assumption becomes fragile in modern hybrid codecs. In HEVC, for instance, the final rate is strongly influenced by decisions made inside Rate-Distortion Optimization (RDO)—such as mode selection, partitioning, and transform choices—which themselves depend on the Lagrange multiplier λ and the cost formulation used during RDO. This creates a classic “chicken and egg” dilemma [58]: to choose an appropriate QP (or λ) that meets the target rate, the encoder needs an accurate prediction of the rate produced by RDO; yet the actual rate cannot be known until RDO has been executed under a specific QP/ λ setting. Consequently, rate control must either (i) approximate RDO outcomes using imperfect models, or (ii) iteratively adjust QP/ λ with additional computation, and both options can degrade either accuracy or efficiency.

Similarly, ρ -domain methods [59] model rate as approximately linear in the fraction of zero transform coefficients. In earlier settings, they were effective because ρ could be mapped to QP in a near one-to-one manner. In HEVC, however, variable block-size

transforms and a highly flexible partitioning structure break this mapping: identical QP values may yield different ρ under different block partitions and transform configurations, and conversely similar ρ can arise from distinct QP choices. As a result, the core $\rho \leftrightarrow$ QP assumption weakens, limiting the reliability of ρ -domain rate control for modern codec structures.

The λ -Domain Rate Control Algorithm

To address the flexibility of HEVC, Li et al. proposed the λ -domain rate control algorithm [7], which has been adopted into the HEVC reference software (HM). This approach shifts the focus from QP to the Lagrange multiplier λ . The core is rooted in the unconstrained Lagrangian optimization problem:

$$J_{opt} = \arg \min_{\theta} (D + \lambda R) \quad (3.4)$$

where D is the distortion, R is the bitrate, and λ is the Lagrange multiplier. Geometrically, λ represents the slope of the tangent line to the convex hull of the operational R-D curve, defined as $\lambda = -\frac{\partial D}{\partial R}$.

Unlike QP, which is discrete, λ is continuous and dictates the trade-off between rate and distortion. The authors discovered that for HEVC, the relationship between λ and bitrate (measured in bits per pixel, bpp) is accurately modeled by a hyperbolic function:

$$\lambda = \alpha \cdot bpp^{\beta} \quad (3.5)$$

where α and β are parameters related to the video source complexity. This model decouples rate estimation from the RDO process: the encoder first determines the optimal λ to satisfy the target bitrate using Eq. (3.5), and then performs RDO. Finally, the QP is

derived from λ using the logarithmic relation $QP = 4.2005 \ln(\lambda) + 13.7122$ [60].

Hierarchical Bit Allocation Strategy

A distinct advantage of the high accuracy of the λ -domain model is its ability to support complex bit allocation strategies. The algorithm implements a unified three-level bit allocation scheme covering Group of Pictures (GOP), Picture, and Basic Unit (BU). To ensure smooth quality transitions and satisfy buffer constraints, the target bits for a GOP are first calculated using a sliding window (SW) mechanism rather than a rigid budget:

$$Target_{GOP} = \frac{R_{PicAvg} \times (N_{coded} + SW) - R_{coded}}{SW} \times N_{GOP} \quad (3.6)$$

where N_{coded} and R_{coded} represent the number of coded pictures and accumulated bits, respectively. Subsequently, these bits are distributed to individual pictures based on their hierarchical importance. Frames in lower temporal layers, which serve as critical references, are assigned higher weights (ω_{Pic}), while higher temporal layer frames receive lower weights:

$$Target_{Pic} = \frac{Target_{GOP} - Coded_{GOP}}{\sum \omega_{Pic}} \times \omega_{CurrPic} \quad (3.7)$$

Finally, within each picture, the remaining bits are allocated to Basic Units (BUs) proportional to their spatial complexity, estimated via the squared Mean Absolute Difference (MAD^2) of the collocated region in the previous frame.

Model Parameter Update

To adapt to non-stationary video content, the parameters α and β in Eq. (3.5) are updated after encoding each unit using the Least Mean Square (LMS) method. The update

rules minimize the error between the logarithmic real λ and the model-predicted λ :

$$\lambda_{comp} = \alpha_{old} b_{pp_{real}}^{\beta_{old}} \quad (3.8)$$

$$\alpha_{new} = \alpha_{old} + \delta_{\alpha} \cdot (\ln \lambda_{real} - \ln \lambda_{comp}) \cdot \alpha_{old} \quad (3.9)$$

$$\beta_{new} = \beta_{old} + \delta_{\beta} \cdot (\ln \lambda_{real} - \ln \lambda_{comp}) \cdot \ln b_{pp_{real}} \quad (3.10)$$

where δ_{α} and δ_{β} are fixed learning rates.

3.2.2 Rate Control Approaches in Neural Video Compression

Rate control for Learned Video Compression (LVC) is an emerging field. Unlike traditional codecs with standardized partitioning, NVCs often rely on end-to-end optimization, making traditional $R - Q$ models difficult to apply directly. Recent works have attempted to bridge this gap through model-based or fully neural approaches. Table 3.1 provides a theoretical comparison of the core mathematical models used in these studies, contrasting the traditional approach with recent neural methods.

Inter-frame Dependency Modeling (Li et al., ICASSP 2022 [10])

To the best of our knowledge, Li et al. [10] proposed the first rate control scheme tailored for Learned Video Compression (LVC). This method is built upon the variable-rate framework by Lin et al. [46], enabling continuous bitrate adjustment via a Lagrange multiplier λ .

The core innovation is the **R-D- λ model**. In contrast to block-level dependency in conventional codecs, the authors model the frame-level dependency via a chain rule of partial derivatives. They introduce a scalar parameter p_t to quantify the linear dependency

Table 3.1: Comparison of Rate Control Schemes: Models, Allocation Strategies, and Updating Mechanisms.

Study	RC Model	Bit Allocation Strategy	Updating Mechanism
[7]	R - λ Model $\lambda = \alpha \cdot R^\beta$	Hierarchical (Sliding Window) Uses a sliding window for GOP-level target, then cascades to pictures using fixed hierarchical weights.	Iterative Update by Real Bits Updates α, β iteratively. Loss: Minimizes error in $\ln \lambda$ domain: $(\ln \lambda_{real} - \ln \lambda_{comp})^2$.
[10]	R - λ Model $\lambda = \alpha \cdot R^\beta$ (Same as [7])	Dependency-based Derived from explicit inter-frame dependency modeling (p -dependency ratio).	Iterative Update by Real Bits Updates α, β and p . Loss: Minimizes error in $\ln R$ domain: $\frac{1}{2}(\ln R_{real} - \ln \hat{R})^2$.
[11]	R - Q_s Model $R = C \cdot Q_s^{-K}$	Hierarchical Directly adopts the sliding window and hierarchical weight mechanism from [7].	Iterative Update by Real Bits Progressive online updating of C, K . Loss: Minimizes error in $\ln R$ domain.
[12]	Neural Network $f(\text{feats}) \rightarrow \lambda$	Neural Network Extracts spatiotemporal features to adaptively allocate bitrates (MiniGoP $\rightarrow$ Frame).	Pre-trained Inference No iterative update. Relies on the generalization of the pre-trained Rate Implementation Network.

between the distortion of consecutive frames:

$$p_t = \frac{\partial D_{t+1}}{\partial D_t}. \quad (3.11)$$

Physically, p_t represents the propagation of quality degradation; a higher value indicates that distortion in the reference frame strongly impacts the subsequent frame.

Staged Parameter Estimation

For rate implementation, the method adopts the hyperbolic model $\lambda = \alpha R^\beta$. To ensure robust estimation, a **staged update algorithm** is employed to decouple the R-D modeling from dependency modeling.

First, α and β are updated via gradient descent to minimize the log-squared rate error loss $l = \frac{1}{2}(\ln \frac{R_{real}}{\hat{R}_i})^2$. The update rules are derived as:

$$\begin{aligned}\alpha_i &= \alpha_{i-1} - \frac{k \cdot \alpha_{i-1} \beta_{i-1}}{2} \ln \frac{R_{real}}{\hat{R}_i}, \\ \beta_i &= \beta_{i-1} - \frac{k \cdot \beta_{i-1}}{2 \ln \hat{R}_i} \ln \frac{R_{real}}{\hat{R}_i},\end{aligned}\tag{3.12}$$

where k is the step size set to 0.5. Subsequently, the dependency parameter p_i is updated in a delayed manner using the observed distortions:

$$p_i \approx \frac{D_i - f_i(R_i) - (D_{i-1} - f_{i-1}(R_{i-1}))}{D_{i-1} - D_{i-2}},\tag{3.13}$$

where $f_i(R_i)$ represents the estimated distortion from the current R-D model. Notably, to prevent parameter instability during the update process, the authors implemented specific heuristic constraints: parameter changes were clamped to 10% for rate increases and 30% for rate decreases.

R-Q Model for NVC (Liao et al., ICASSP 2024 [11])

Addressing the limitations of applying fixed λ models to variable-rate networks, Liao et al. [11] proposed a Rate-Quality ($R - Q_s$) based control scheme. This approach is designed as a "plug-in" strategy for NVC frameworks where bitrate is regulated by a latent scaling parameter Q_s . As illustrated in Fig. 3.9, this scheme operates in a plug-in fashion, decoupling the rate estimation from the internal network architecture.

R-Q Model and Parameter Update

Unlike the hyperbolic $R - \lambda$ relation, this method establishes a power-law relationship

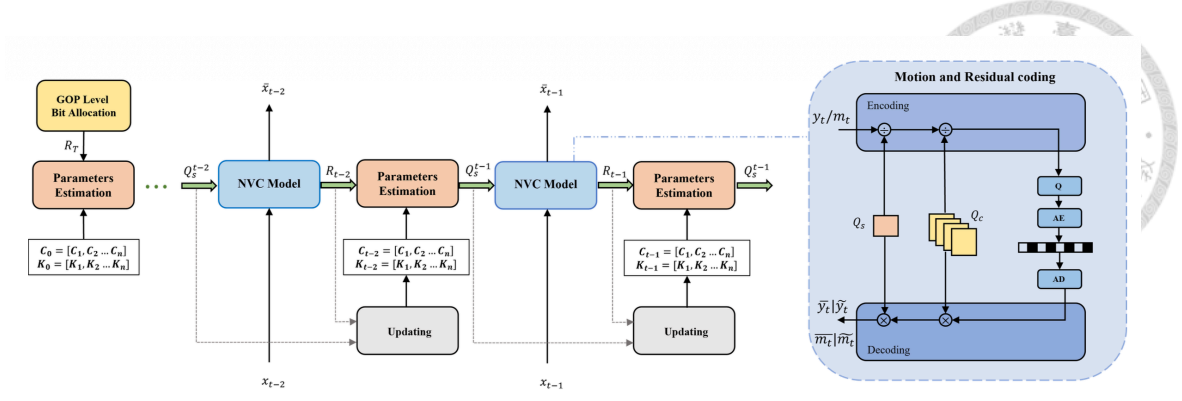


Figure 3.9: The framework of the Rate-Quality ($R - Q_s$) based rate control scheme proposed by [11].

between the target bitrate R and the scaling factor Q_s , formulated as:

$$R = C \cdot Q_s^{-K}, \quad (3.14)$$

where C and K are content-dependent parameters. To adapt to non-stationary video content, C and K are updated frame-by-frame via gradient descent in the logarithmic domain.

The update rules are defined as:

$$\begin{aligned} C'_i &= C_i - \eta \cdot (\ln \hat{R}_i - \ln R_{real}) \cdot C_i, \\ K'_i &= K_i - \mu \cdot (\ln \hat{R}_i - \ln R_{real}) \cdot \ln Q_{s,i}, \end{aligned} \quad (3.15)$$

where $\hat{R}_i$ is the predicted rate, R_{real} is the actual rate, and η, μ are step sizes.

Hierarchical Structure Preservation

A critical observation in this work is that traditional frame-level allocation often disrupts the hierarchical quality structure naturally learned by NVCs (where specific frames carry learned importance weights ω_i). By controlling the global scaling factor Q_s through the $R - Q_s$ model rather than forcing a rigid bit distribution derived from inter-frame dependency models, this method allows the network to implicitly allocate bits according to its internal weights, thereby minimizing R-D performance degradation.

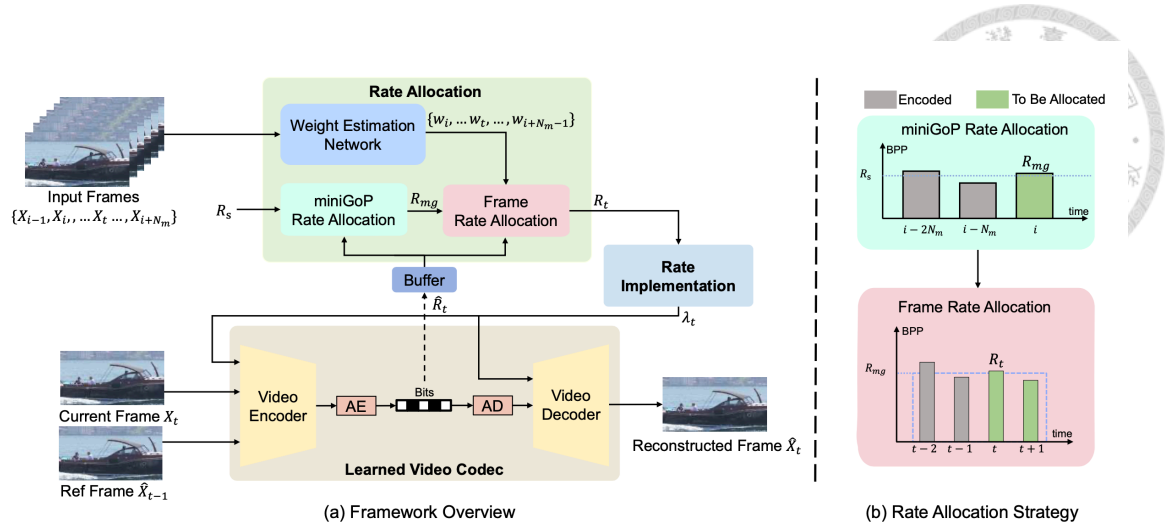


Figure 3.10: **Overview of the fully neural rate control system proposed by Zhang et al. [12].** (a) The framework utilizes a Weight Estimation Network for hierarchical allocation and a Rate Implementation Network for parameter prediction. (b) Visualization of the hierarchical allocation strategy, distributing bits from miniGoP level down to frame level based on feature importance.

Fully Neural Rate Control System (Zhang et al., ICLR 2024 [12])

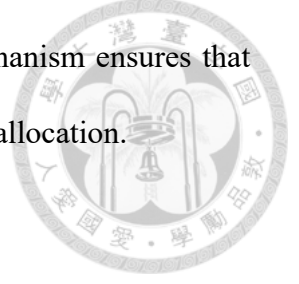
While previous works adapted traditional empirical models, Zhang et al. [12] argue that fixed mathematical forms struggle to capture the non-linear rate-distortion characteristics of deep video compression. To address this, they proposed the first fully neural network-based rate control system, as illustrated in Fig. 3.10(a).

Two-Level Rate Allocation Strategy

To overcome the limitations of uniform allocation, the authors introduced a hierarchical strategy (Fig. 3.10(b)) comprising **miniGoP-level** and **Frame-level** allocation. Initially, the target bits for a miniGoP (R_{mg}) are determined via a sliding window. Subsequently, a **Weight Estimation Network** extracts spatiotemporal features to assign an importance weight w_t to each frame. By leveraging these weights, the system implicitly models inter-frame dependency, calculating the target bitrate R_t as:

$$R_t = \frac{R_{mg} - \hat{R}_{mg}}{\sum_{j=t}^{i+N_m-1} w_j} \times w_t, \quad (3.16)$$

where $\hat{R}_{mg}$ denotes the used bits in the current miniGoP. This mechanism ensures that reference frames with higher impact features receive a larger budget allocation.



Rate Implementation via Feature Regression

Instead of iterative search algorithms, this method employs a **Rate Implementation Network** to directly map the target bitrate R_t to the encoding parameter λ_t . As shown in the blue block of Fig. 3.10(a), this mapping is formulated as a regression problem conditioned on content and historical statistics. A critical normalization step modulates the feature vector $\vec{V}_R$ derived from the input bitrate:

$$\vec{V}'_R = \frac{\vec{V}_R - \mu}{\theta}, \quad (3.17)$$

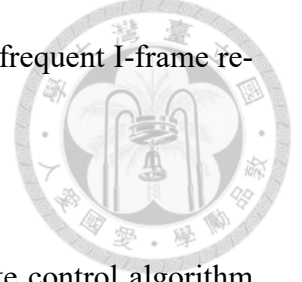
where (μ, θ) are normalization parameters fused from image and historical features. This design enables precise λ prediction in a single forward pass, significantly reducing encoding latency.

3.2.3 Comparative Analysis and Limitations

While the reviewed methods show progress, a systematic analysis reveals significant gaps when applying them to real-time scenarios. Table 3.2 provides a granular breakdown of the experimental settings used in prior works. Specifically, we focus on three critical parameters often overlooked in literature but vital for reproducibility and practical deployment:

1. **Intra Period (N_{intra}):** Indicates the frequency of I-frames. For real-time applications like cloud gaming, an "infinite GOP" setting ($N_{intra} = -1$) is typically

required to minimize latency spikes, yet many studies rely on frequent I-frame re-sets.



2. **Bit Allocation Unit:** Defines the granularity at which the rate control algorithm allocates bits (often referred to as the GOP unit in RC contexts). The absence of this specification makes reproducing the bit allocation logic nearly impossible.
3. **Test Frames:** The duration of test sequences. Short sequences risk "overfitting" to a single scene's complexity. Algorithms must be tested on longer durations to verify stability during scene cuts and distinct complexity shifts.

Table 3.2: Comparison of Experimental Settings and Practical Limitations in Existing Studies.

Study	Target Codec	N_{intra}	GOP	Frames	Critical Limitations
Li et al. [7]	HEVC (HM 8.0)	-1	4	-1	Validated on traditional codecs; fails to address the feature-domain characteristics of NVC.
Li et al. [10]	DVC + LVC	12	12	N/A	Uses extremely short GOPs, masking potential error propagation issues inherent to infinite-GOP settings.
Liao et al. [11]	DVC, DMVC, DCVC-HEM, DCVC-DC	N/A	N/A	192	Critical bit allocation unit is missing; implementation details are ambiguous regarding Q_s usage.
Zhang et al. [12]	DVC, FVC, DCVC, AlphaVC	100	4	100	Requires complex three-stage training; high computational overhead unsuitable for real-time constraints.

Note: "N/A" indicates data not available in the original paper. N_{intra} denotes the I-frame interval. A value of -1 indicates the full sequence length (for Frames) or a single I-frame setting (for N_{intra}).

Gap between Academic Prototypes and Practical Deployment

A critical challenge in current NVC rate control research is the disconnect between evaluation protocols and real-world application requirements. As detailed in Table 3.2, many existing studies adopt settings that diverge significantly from industrial standards. For instance, Li et al. [10] evaluated their model on short sequences ($N_{intra} = 12$). While such settings were acceptable for early-stage NVCs—which were primarily proofs of concept—they obscure stability issues that only manifest in long-duration streaming. Modern low-delay applications require stability over infinite GOPs ($N_{intra} = -1$), where error propagation can accumulate indefinitely if not properly managed.

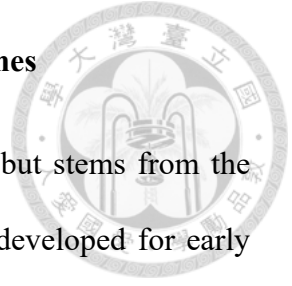
Furthermore, the duration of test sequences (Test Frames) plays a pivotal role. Short clips often lack significant scene changes, allowing rigid algorithms to perform deceptively well. Our analysis suggests that true robustness can only be verified when the rate controller is forced to adapt to drastic variance in scene complexity over extended periods.

Ambiguity in Parameter Initialization

In addition to inconsistent protocols, the literature often lacks transparency regarding initialization strategies and hyperparameter sensitivity.

- **Initialization:** Improper starting values for model parameters (α, β) can cause severe bitrate fluctuations in the initial phase, leading to buffer violations before the model converges.
- **Adaptability:** Algorithms using fixed learning rates often fail to adapt to the variance in scene complexity found in real-time scenarios. A rate too small causes sluggish adaptation during scene cuts, while a rate too large induces oscillation.

The Necessity of Aligning Rate Control with Mature NVC Baselines



The most significant gap, however, is not merely algorithmic but stems from the evolution of NVC capability. Previous rate control methods were developed for early NVC architectures that struggled to compete with traditional codecs in terms of efficiency and complexity. In contrast, **DCVC-RT** has emerged as a landmark architecture, achieving compression performance and runtime efficiency that align with practical deployment needs. This shift necessitates a re-evaluation of rate control strategies:

1. **From Experimental to Practical Baselines:** Early rate control schemes were often tested on NVCs that were not yet viable for production. As DCVC-RT has effectively bridged the gap towards practical application—becoming a strong candidate for large-scale deployment—its associated rate control modules must essentially “graduate” from academic experimentation to robust, industrial-grade reliability. Future work on such a mature codec implies a requirement for stability and precision comparable to commercial standards.
2. **The Value of Architecture-Specific Optimization:** While tailoring a rate control algorithm to a specific NVC architecture (like DCVC-RT) inherently involves a risk of overfitting to that architecture’s specific modules, this trade-off is justified by the architecture’s maturity. Much like optimizing rate control for H.265 remains highly valuable despite the existence of H.266, refining control mechanisms for a deployment-ready NVC like DCVC-RT possesses significant practical value. The focus is no longer just on proving that neural rate control is possible, but on ensuring it operates seamlessly within a high-performance framework that is likely to see actual adoption.

In conclusion, while DCVC-RT sets a new benchmark for compression efficiency, existing rate control algorithms lag behind its practical readiness. The lack of a rate control scheme that matches the utility and stability of DCVC-RT creates a bottleneck for its deployment. Therefore, this thesis focuses on developing a robust rate control mechanism specifically aligned with DCVC-RT, aiming to complete the final puzzle piece for a truly practical neural video coding system.





Chapter 4

Proposed Method: Adaptive

State-Space Design for Real-Time NVC

Rate Control

4.1 System Overview

In this study, we treat the Neural Video Compressor (NVC) system, specifically DCVC-RT, as a non-linear time-variant system. To facilitate the derivation of the rate control algorithm, the encoder is abstracted as a Single-Input Single-Output (SISO) black-box model. As shown in Fig. 4.1, the system operates as a typical closed-loop control system.

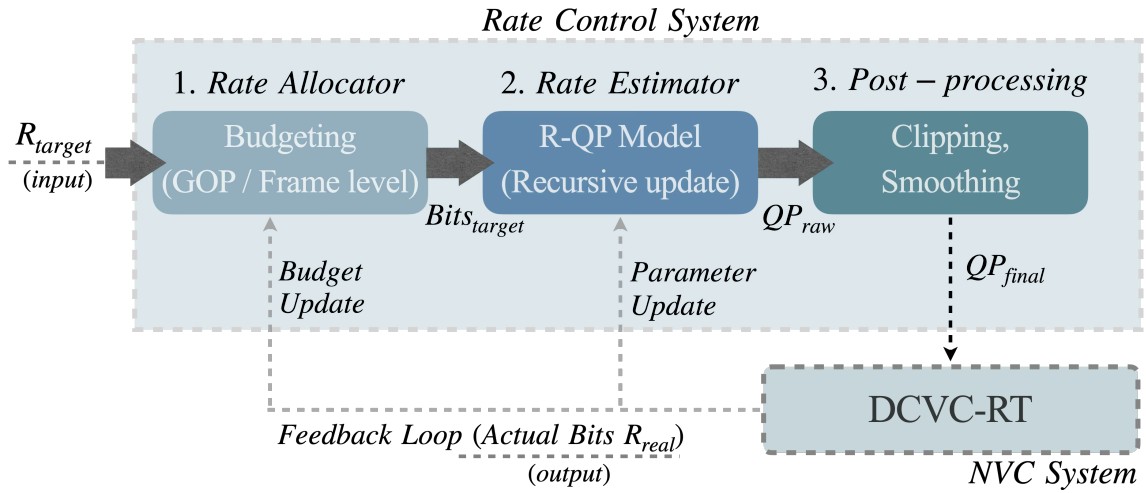


Figure 4.1: Overall architecture of the proposed rate control system. The DCVC-RT is treated as a black-box model. The controller determines the QP_{final} based on the target bitrate and updates its internal model using the feedback loop after encoding.

Let t denote the index of the video frame and X_t be the input raw image frame. For each frame, the objective of the Rate Control System is to determine a final Scaling Parameter (QP_{final}) based on the input target bitrate R_{target} , such that the actual number of bits generated by the encoder, R_{real} , is as close as possible to the budget. The input-output

relationship of the system can be expressed as:

$$R_{real,t}, \hat{X}_t = \mathcal{F}_{enc}(X_t, QP_{final,t} \mid \Theta_{net}) \quad (4.1)$$

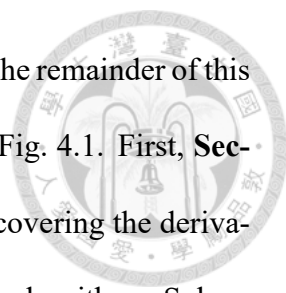


where $\mathcal{F}_{enc}(\cdot)$ represents the encoding process of DCVC-RT, Θ_{net} denotes the fixed, pre-trained weights of the neural network, and $\hat{X}_t$ is the reconstructed image. Since we cannot modify Θ_{net} in real-time during the encoding process, QP becomes the sole control variable.

As illustrated in Fig. 4.1, our rate control system comprises three cascaded modules:

1. **Rate Allocator:** Responsible for **Budgeting** at both the Group of Pictures (GOP) and Frame levels. It translates the global bitrate target R_{target} into a specific bit budget for the current frame, denoted as $Bits_{target}$.
2. **Rate Estimator:** Maintains an internal **R-QP Model** to describe the relationship between quantization and bitrate. It takes the allocated $Bits_{target}$ as input and predicts the baseline quantization parameter, QP_{raw} . This module employs a recursive update mechanism to adapt to video content changes.
3. **Post-processing:** Ensures the validity and stability of the control signal. It applies **Clipping** and **Smoothing** to the raw prediction QP_{raw} to generate the final control command QP_{final} , preventing buffer overflow and temporal flickering.

After the encoding of frame t is completed, the system utilizes a **Feedback Loop** to feed the actual generated bits R_{real} back to the controller. This feedback triggers the **Update** of the budget status in the Rate Allocator and the **Parameter Update** in the Rate Estimator, thereby optimizing the decision-making for frame $t + 1$.



To facilitate a systematic exposition of the proposed framework, the remainder of this chapter is organized corresponding to the key modules illustrated in Fig. 4.1. First, **Sections 4.2 and 4.3** detail the core mechanism of the **Rate Estimator**, covering the derivation of the Log-linear R-Q model and the adaptive Dual-State RLS algorithm. Subsequently, **Section 4.4** elaborates on the **Rate Allocator**, focusing on the sliding window budgeting and hierarchical weight distribution. Finally, **Section 4.5** addresses the **Post-processing** and control logic, specifically the dual-smoothing mechanisms designed to ensure system stability and safety.

4.2 R-Q Modeling and Empirical Analysis

Empirical R-Q Analysis via Exhaustive Search

To achieve precise rate control in a closed-loop system, the primary task is to establish a mathematical model between the Quantization Parameter (QP) and bits per pixel (R_{bpp}). Unlike traditional video coding standards which often model in the λ -domain, we aim to identify the characteristic that best represents the essence of the DCVC-RT architecture. To eliminate interference from prediction errors and obtain the ground truth relationship, we employed an **Exhaustive Search** strategy.

During the encoding of test sequences, we forced the encoder to traverse all allowed integer QP values ($QP \in [0, 63]$), thereby obtaining the Ground Truth R-Q Curve for each frame globally. Based on these high-density data points, we evaluated four common candidate models:

- Linear Model: $QP = a \cdot R_{bpp} + b$
- Power Model: $QP = a \cdot R_{bpp}^b$ (equivalent to $R = \alpha \cdot QP^\beta$ [11])



- Exponential Model: $QP = a \cdot e^{b \cdot R_{bpp}}$
- Logarithmic Model: $QP = a \cdot \ln(R_{bpp}) + b$

Fig. 4.2 presents the curve fitting results using the exhaustive data. Experimental results show that the **Logarithmic Model** exhibits the highest goodness-of-fit (R^2 score) across test frames. The Linear model tends to underestimate QP in the low bitrate range, while the Power model deviates significantly in the high bitrate range. In contrast, the Log model accurately captures the characteristic of DCVC-RT where **”bitrate increases exponentially as QP increases.”**

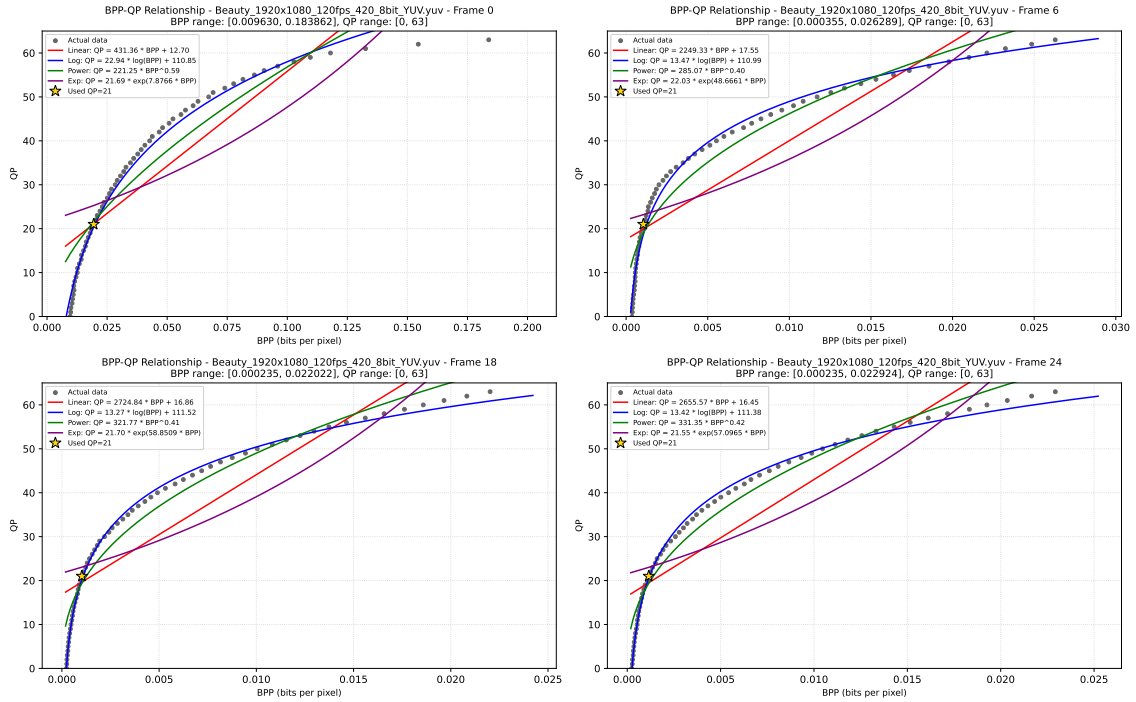


Figure 4.2: Empirical R-Q curve fitting based on exhaustive QP search data. The blue curve (Logarithmic model) demonstrates the highest fidelity to the actual data points across the entire bitrate range compared to other candidates.

Parameter Stability Analysis

Beyond static fitting accuracy, the *temporal stability* of model parameters is crucial for the design of a stable control algorithm. If model parameters fluctuate violently be-

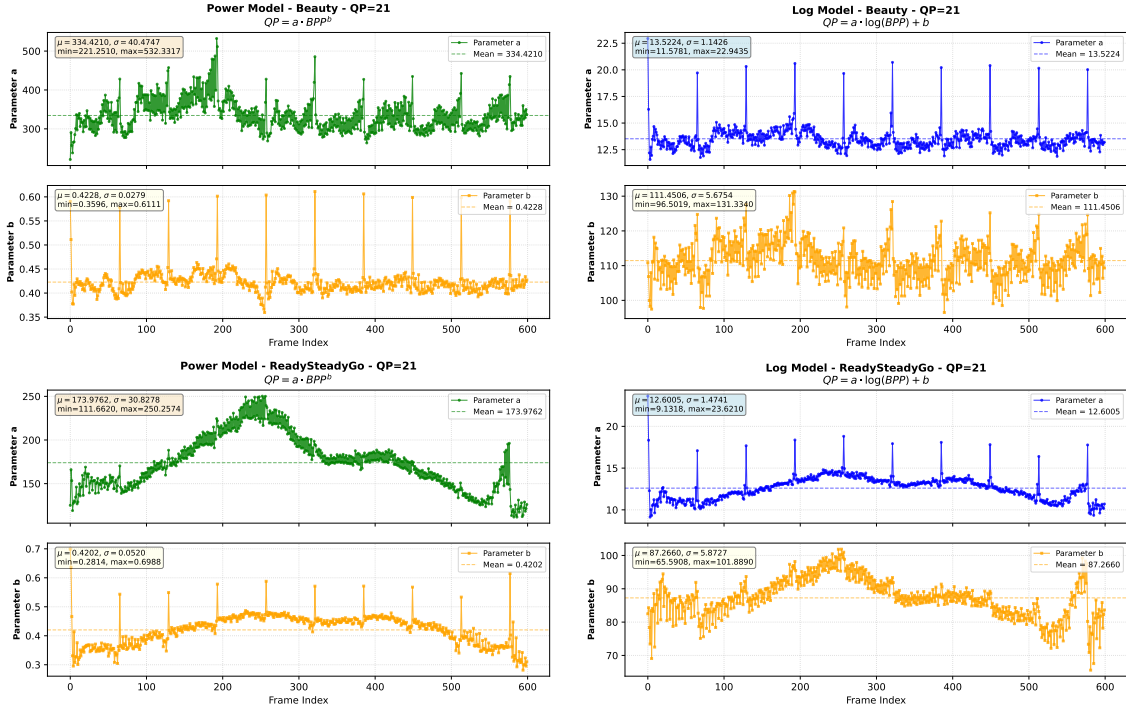


Figure 4.3: Comparison of parameter evolution between Power Model (Left) and Log Model (Right). Top row: *Beauty* (Low SI/TI); Bottom row: *ReadySteadyGo* (High TI). The Log Model parameters (α) exhibit stable periodicity within a predictable range (10 ~ 20), whereas the Power Model parameters fluctuate wildly (100 ~ 550) and lack clear patterns.

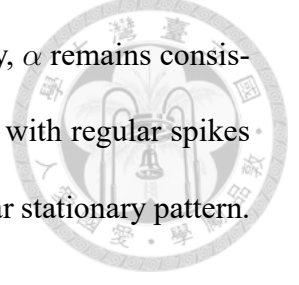
tween adjacent frames, the controller may suffer from overshoot or instability.

As illustrated in Fig. 4.3, we compared the parameter evolution of the Power Model versus our Logarithmic Model over 600 frames. We selected two representative sequences with distinct characteristics: *Beauty* (Low SI/TI) and *ReadySteadyGo* (High SI/TI).

The comparison reveals significant differences in parameter dynamics:

- **Power Model (Left):** The multiplier parameter a exhibits extreme variance and unpredictability. For *Beauty*, a fluctuates between 220 and 530; for *ReadySteadyGo*, it drifts between 120 and 240. Such a wide and erratic dynamic range places a heavy tracking burden on the online estimator, making it prone to divergence.
- **Log Model (Right):** In contrast, our model's slope parameter α (blue line) demon-

strates remarkable stability. Regardless of sequence complexity, α remains consistently within the range of $[10, 20]$. It exhibits a stable baseline with regular spikes corresponding to the periodic Refresh Frames, displaying a clear stationary pattern.



Model Linearization and Objective Function

Based on the empirical evidence and stability analysis above, we officially adopt the Log-linear R-Q Model:

$$QP_t = \alpha_t \cdot \ln(R_{bpp,t}) + \beta_t \quad (4.2)$$

This finding distinguishes our work from recent NVC rate control research which often utilizes Power Models (requiring non-linear optimization). Our model possesses a critical mathematical advantage: **Parameter Linearization**.

The equation can be viewed as a standard linear equation $y_t = \theta_t^T \phi_t$, where the target variable y_t , parameter vector θ_t , and feature vector ϕ_t are defined as:

$$y_t = QP_t, \quad \theta_t = \begin{bmatrix} \alpha_t \\ \beta_t \end{bmatrix}, \quad \phi_t = \begin{bmatrix} \ln(R_{bpp,t}) \\ 1 \end{bmatrix} \quad (4.3)$$

We define the rate control problem as an optimization problem minimizing the squared prediction error. The instantaneous loss function is:

$$\mathcal{L}_t(\theta_t) = (QP_{real,t} - \theta_t^T \phi_t)^2 \quad (4.4)$$

This linear structure allows us to avoid complex non-linear solvers and instead employ efficient recursive algorithms for online estimation.



4.3 Adaptive Parameter Estimation

4.3.1 Recursive Least Squares (RLS) Algorithm

Due to the non-stationarity of video content, scene changes or fluctuations in complexity cause the optimal model parameters α_t and β_t to vary over time. To adapt to this time-variant characteristic, we leverage the **Recursive Least Squares (RLS)** algorithm.

The RLS algorithm is particularly suitable here because, as established in the previous section, our system model is linear in its parameters. RLS minimizes the cumulative weighted error:

$$\hat{\theta}_t = \arg \min_{\theta} \sum_{i=0}^t \lambda^{t-i} \mathcal{L}_i(\theta) \quad (4.5)$$

where $\lambda \in (0, 1]$ is the forgetting factor. This factor balances the importance of historical data against current data, enabling the model to adapt quickly to scene changes.

Fig. 4.4 illustrates the update mechanism. The specific steps for each frame t are detailed as follows:

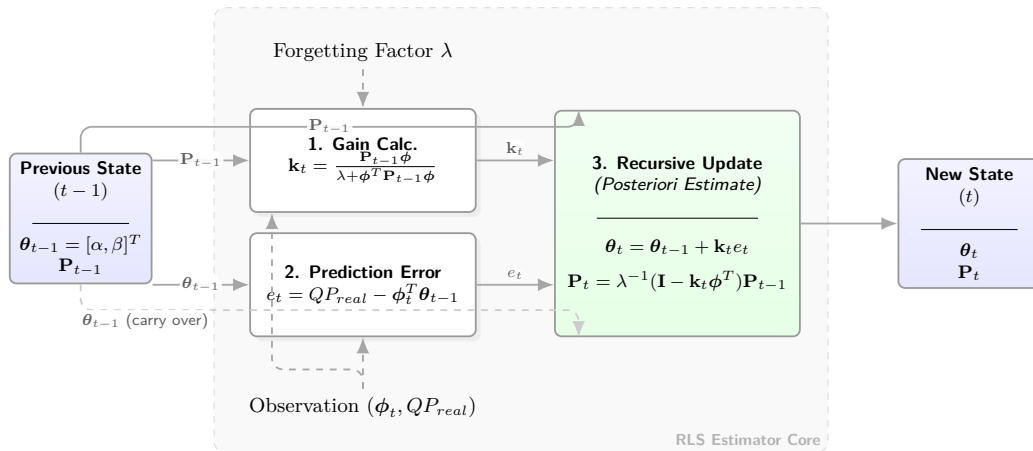


Figure 4.4: Internal mechanism of the RLS Estimator. The model updates its parameter state θ and covariance matrix P recursively based on the prediction error of the previous frame.

1. **Calculate Gain Vector:** Based on the previous covariance matrix $\mathbf{P}_{t-1}$ and current feature vector ϕ_t , compute the Kalman Gain $\mathbf{k}_t$. This gain determines how much the current prediction error should influence the parameter update:

$$\mathbf{k}_t = \frac{\mathbf{P}_{t-1}\phi_t}{\lambda + \phi_t^T \mathbf{P}_{t-1} \phi_t} \quad (4.6)$$

In our implementation, we set $\lambda = 0.995$ for general P-frames to maintain stability while allowing adaptation.

2. **Calculate Prediction Error (Priori Error):** Compute the error e_t between the actually observed QP_{used} and the value predicted by the previous state:

$$e_t = QP_{used,t} - \phi_t^T \theta_{t-1} \quad (4.7)$$

3. **Update Model Parameters:** Correct the parameter estimate θ_t using the calculated gain and error:

$$\theta_t = \theta_{t-1} + \mathbf{k}_t e_t \quad (4.8)$$

4. **Update Covariance Matrix:** Update the inverse correlation matrix $\mathbf{P}$ for the next iteration to reflect the reduced uncertainty:

$$\mathbf{P}_t = \frac{1}{\lambda} (\mathbf{P}_{t-1} - \mathbf{k}_t \phi_t^T \mathbf{P}_{t-1}) \quad (4.9)$$

Through this recursive process, we obtain the optimal estimate $\theta_t = [\alpha_t, \beta_t]$ immediately after each frame is encoded.

4.3.2 Comparison with Gradient-based Methods

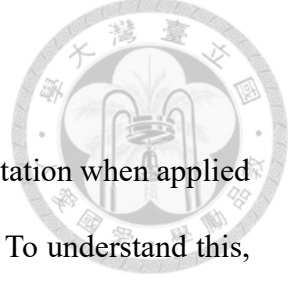
While gradient-based methods such as Least Mean Squares (LMS) or Stochastic Gradient Descent (SGD) are commonly used for adaptive filtering, we selected RLS for the DCVC-RT rate control system due to its superior convergence properties in non-stationary environments.

As summarized in Table 4.1, although gradient-based methods offer extremely low computational complexity, they rely solely on the instantaneous error gradient. This often results in slow convergence and high sensitivity to the learning rate (step-size). In contrast, RLS utilizes **second-order statistics** via the covariance matrix **P**. This allows it to "whiten" the input data, leading to extremely fast convergence (typically within 1-2 frames in our experiments).

Although RLS theoretically has a higher computational cost ($O(N^2)$) compared to LMS ($O(N)$), our model dimension is small ($N = 2$, corresponding to parameters α and β). Therefore, the overhead is negligible, making RLS the optimal choice for our specific application where stability and fast tracking are paramount.

Table 4.1: Comparison between Gradient-based (LMS/SGD) algorithms and the proposed RLS Filter for Rate Control.

Feature	Gradient-based (SGD)	RLS Filter (Proposed)
Convergence Speed	Slow; depends heavily on the learning rate.	Extremely Fast; typically converges within 1-2 frames.
Information Utilization	Uses only the current error gradient (First-order).	Uses second-order statistics (Covariance Matrix P).
Robustness	Sensitive to hyper-parameter tuning.	More robust via the Forgetting Factor (λ).
Complexity	Extremely low ($O(N)$).	Low for our 2×2 model ($O(N^2)$); negligible overhead.



4.3.3 Challenge: Vulnerability to Refresh Frames

While RLS provides fast convergence, it possesses a critical limitation when applied to the DCVC-RT architecture: **Vulnerability to Systemic Outliers**. To understand this, we must first analyze the "Refresh Mechanism" inherent to the codec.

Mechanism of Periodic Refresh To mitigate error propagation in the long prediction chain, DCVC-RT employs a periodic feature refresh strategy. As illustrated in Fig. 4.5, the temporal context generation module ($f_{Tcontext}$) contains a **Refresh Control** switch that alternates between two distinct input paths:

1. **Standard Path (Yellow):** In normal P-frames, the temporal context is derived from the propagated high-dimensional latent feature F_{t-1} . This path contains rich semantic history but accumulates quantization errors over time.
2. **Refresh Path (Green):** When the refresh counter triggers (e.g., every 32 frames), the switch disconnects the feature propagation. Instead, it forces the model to extract features solely from the reconstructed pixel domain image $\hat{x}_{t-1}$.

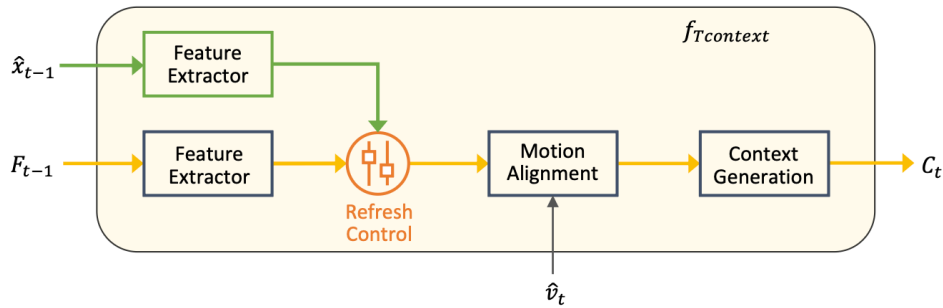


Figure 4.5: The internal structure of the temporal context module ($f_{Tcontext}$). The **Refresh Control** determines whether the context is generated from the propagated feature F_{t-1} (Standard Mode) or the reconstructed image $\hat{x}_{t-1}$ (Refresh Mode). This path switching fundamentally alters the signal characteristics.

Since $\hat{x}_{t-1}$ (Pixel Domain) carries significantly different information density and sta-

tistical properties compared to F_{t-1} (Latent Feature Domain), the R-Q relationship undergoes an abrupt structural change during these frames.



The Phenomenon of "Model Shattering" From the perspective of a single-state RLS estimator, this structural change acts as a massive outlier. As illustrated in Fig. 4.6, applying a standard RLS filter results in a phenomenon we term **"Model Shattering."** The process occurs as follows:

1. **Outlier Impact:** When the Refresh Path is activated, the model generates a massive instantaneous prediction error (e_t) because the previous parameters θ_{t-1} were learned from the Standard Path statistics.
2. **Over-Reaction:** Since RLS attempts to minimize the squared error $\sum e_t^2$, it drastically adjusts the parameters α and β to accommodate this single outlier.
3. **Covariance Corruption:** The internal covariance matrix $\mathbf{P}_t$, which represents the confidence of the estimator, is corrupted by the non-representative feature vector of the refresh frame.
4. **Memory Effect:** Due to the recursive nature, this corrupted state propagates to subsequent frames. The estimator suffers from a "hangover" effect, leading to slow recovery and instability in the following standard P-frames.

4.3.4 Proposed Solution: Dual-State RLS

To address the model shattering caused by refresh frames, we propose a **Dual-State RLS** strategy. The core idea is to prevent "Parameter Poisoning" by isolating the R-D modeling into two distinct states based on frame characteristics.

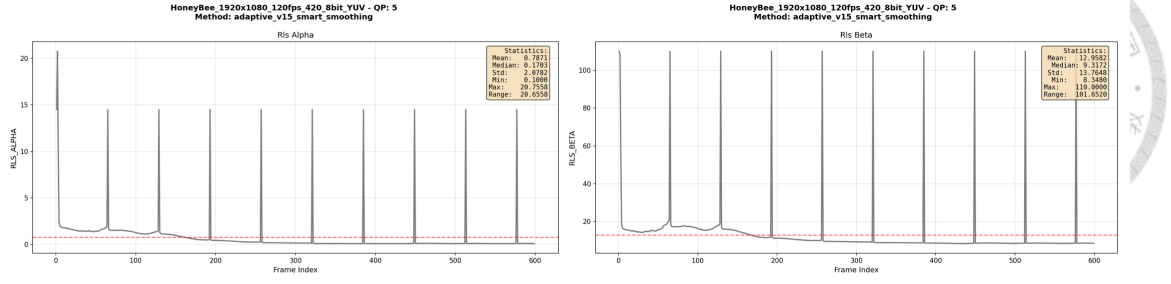


Figure 4.6: The "Model Shattering" phenomenon caused by single-state RLS. Due to the massive outliers introduced by Refresh Frames, the estimator's covariance matrix is irreversibly corrupted. This causes the model parameters (Alpha and Beta) to collapse towards zero and fail to recover, losing their ability to track the true R-Q characteristic of standard frames.

We define two independent RLS estimators:

- **State A (Inter-State):** Updates only on standard P-frames (high temporal dependency).
- **State B (Refresh-State):** Updates only on Refresh frames (low temporal dependency).

By temporally decoupling the states, the covariance matrix of the Inter-State remains pristine and unaffected by the statistics of Refresh frames.

Experimental Results: Fig. 4.7 demonstrates the effectiveness of this approach. In the single-state baseline (from the shattering example), parameters would often collapse or exhibit high variance ($\sigma \approx 20.7$). With the Dual-State strategy, the parameters α and β no longer collapse. Instead, they exhibit **stable oscillations** within physically meaningful bounds. The standard deviation of the parameter α is significantly reduced, indicating a robust estimation process that generalizes well across the entire video sequence.

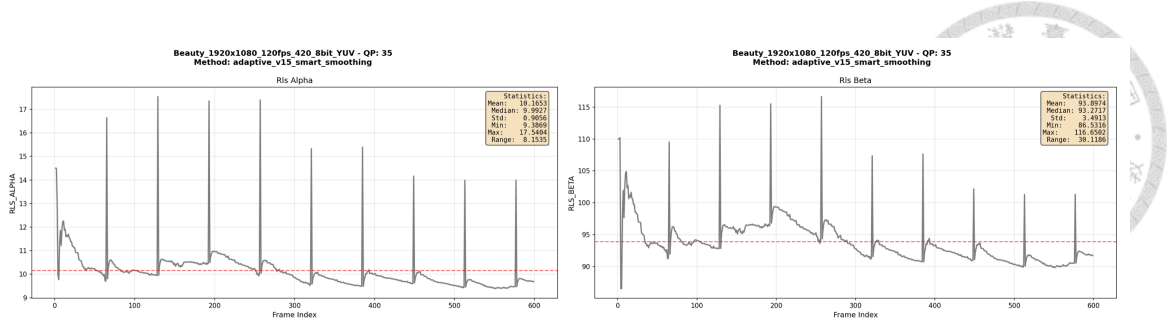


Figure 4.7: Performance of the Dual-State RLS. By decoupling the refresh frames, the parameters maintain stability without collapsing to zero. The curves show stable oscillations, confirming that the estimator correctly distinguishes between the two encoding modes.

4.3.5 Parameter Stabilization (First-Stage Smoothing)

While the Dual-State RLS responds effectively to system changes, the raw estimated parameters $\hat{\theta}_t$ may still exhibit high-frequency jitter due to inherent measurement noise in the neural network’s feedback. To further stabilize the controller, we implement a **Parameter Smoothing** layer.

Before the parameters are used for prediction in the next frame, we apply an Exponential Moving Average (EMA) to the RLS output:

$$\theta_{ema,t} = \eta \cdot \theta_{ema,t-1} + (1 - \eta) \cdot \hat{\theta}_t \quad (4.10)$$

where η is the smoothing factor (typically set to 0.7) and $\hat{\theta}_t$ is the instantaneous estimate from the RLS update. The calculation of the base quantization parameter QP_{raw} is then performed using these smoothed parameters θ_{ema} . This creates a "flywheel effect," preventing the controller from reacting too aggressively to single-frame noise while maintaining the general trend tracking capability of RLS.



4.4 Bitrate Allocation Strategy

4.4.1 GOP-based Allocation with Sliding Window

To ensure the stability of the output bitrate while accommodating the specific characteristics of Neural Video Coding (NVC), we implement a hybrid allocation strategy that combines a sliding window for long-term traffic smoothing with dynamic rebalancing within the Group of Pictures (GOP).

As defined in our implementation, the base bit budget for the current GOP is calculated using a sliding window mechanism. This effectively smoothes out short-term fluctuations caused by scene complexity changes. The average target bits per frame, R_{avg} , is derived as follows:

$$R_{avg} = \frac{R_{tgt} \cdot (N_{coded} + SW) - R_{cum}}{SW} \quad (4.11)$$

where R_{tgt} is the user-defined target bitrate, N_{coded} is the number of encoded frames, R_{cum} is the cumulative actual bits consumed, and SW is the sliding window size (typically 40 frames). This formula ensures that if previous frames under-used the budget, the surplus is distributed to future frames, and vice versa.

4.4.2 Hierarchical Quality Structure and Feature Propagation

Rationale for Periodic "Neural Anchors"

In traditional video coding (e.g., H.264), I-frames are inserted periodically to stop error propagation. However, in the DCVC-RT architecture, inserting frequent I-frames is

prohibitively expensive in terms of bitrate. Instead, the system relies on the continuous propagation of latent features F_t to serve as temporal contexts for future frames.

This propagation mechanism introduces a unique challenge: **Feature Degradation**.

As the distance from the last refresh point increases, the quality of the propagated feature gradually decays due to accumulated quantization noise and motion alignment errors. If the feature quality drops below a critical threshold, the neural decoder struggles to reconstruct the next frame effectively, leading to a "drifting" visual artifacts.

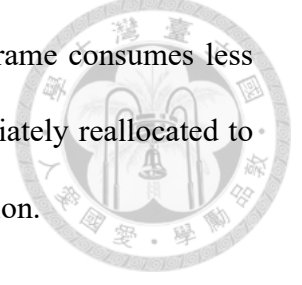
To mitigate this, we adopt a Hierarchical Quality Structure by applying a periodic weight pattern Ω . We introduce the concept of "**Neural Anchors**". By assigning higher weights (and thus **higher QPs**) to specific frames periodically (e.g., every 4 or 8 frames), we force the encoder to generate high-fidelity reconstructed features. These high-quality features act as anchors, effectively "sharpening" the temporal context. This allows subsequent frames—even those encoded with higher QPs—to reference a cleaner history, thereby improving the overall Rate-Distortion performance of the entire GOP.

4.4.3 Dynamic GOP Rebalancing

To strictly adhere to the GOP budget while respecting the weight pattern, we employ a dynamic rebalancing mechanism. Within a GOP, the target bitrate for the current frame t , denoted as R_{pic} , is calculated based on the *remaining* GOP budget R_{rem} and the *remaining* weights $\Sigma\omega_{rem}$:

$$R_{pic} = \max \left(R_{floor}, R_{rem} \cdot \frac{\omega_t}{\sum_{i=t}^N \omega_i} \right) \quad (4.12)$$

This dynamic approach ensures that if a high-weight anchor frame consumes less than expected (e.g., due to a static scene), the saved bits are immediately reallocated to the remaining frames in the same GOP, maximizing resource utilization.



4.5 Dynamic Smoothing and Control

4.5.1 Smart Output Smoothing (Second-Stage Smoothing)

After the RLS model predicts the raw quantization parameter (QP_{raw}) using the smoothed parameters θ_{ema} , we apply a final safety mechanism known as **Smart Output Smoothing**.

Directly applying the predicted QP can sometimes lead to visual flickering, especially when the model tries to correct for minor bitrate deviations too aggressively. However, excessive smoothing can lead to buffer overflows during scene cuts. To resolve this trade-off, we propose a dynamic mechanism based on the real-time Error Ratio (E_t).

We define the error ratio as the normalized deviation of the actual bitrate from the target over a short window:

$$E_t = \frac{\text{mean}(E_{window})}{R_{tgt}} \quad (4.13)$$

The final QP is determined by dynamically adjusting the smoothing factor γ :

$$QP_{smooth,t} = \gamma \cdot QP_{smooth,t-1} + (1 - \gamma) \cdot QP_{raw,t} \quad (4.14)$$

The logic for γ is adaptive:

- **Stability Mode** ($E_t < \delta$): When the controller is tracking well (e.g., error ratio

$< 10\%$), we set a high smoothing factor (e.g., $\gamma = 0.8$). This acts as a low-pass filter, ensuring gradual QP transitions and stable visual quality.

- **Emergency Response ($E_t \geq \delta$):** If the error spikes (e.g., due to a sudden scene cut or complex motion), γ is set to 0.0. The controller bypasses the smoother, allowing the RLS prediction to instantly react and bring the bitrate back to the target level.

4.5.2 Algorithm Summary

The complete proposed method, integrating the sliding window allocation, dynamic GOP rebalancing, and the Double-Smoothing RLS (Parameter EMA + Output Smoothing), is summarized in Algorithm 1.



Algorithm 1 Proposed Dual-State RLS with Double-Smoothing Strategy

Input: Target R_{tgt} , Weights Ω , Window SW , GOP N , Margin δ

Hyperparams: Output Smooth γ , Param EMA η , RLS Forgetting λ

Output: Final QP_t for current frame t

1. System Initialization

Initialize Dual RLS States: $\hat{\theta}_{Normal}, \hat{\theta}_{Refresh}$, Covariance $\mathbf{P} \leftarrow 100\mathbf{I}$

Initialize Smoothed Params: $\theta_{ema} \leftarrow \hat{\theta}_{Normal}$

Initialize Allocator: $R_{cum} \leftarrow 0, N_{coded} \leftarrow 0$, Error Queue $E \leftarrow \emptyset$

2. Processing Loop

for each frame t in video sequence **do**

// Step A: Sliding-Window & GOP Budgeting

if Start of new GOP **then**

$R_{avg} \leftarrow \max(R_{floor}, [R_{tgt} \cdot (N_{coded} + SW) - R_{cum}]/SW)$

$R_{rem} \leftarrow R_{avg} \cdot N; \quad \Sigma\omega_{rem} \leftarrow \sum_{i=1}^N \omega_i$

end if

// Step B: Bitrate Allocation

$R_{pic} \leftarrow \max(R_{floor}, R_{rem} \cdot (\omega_t/\Sigma\omega_{rem}))$

// Step C: Model Prediction (Using Smoothed Parameters)

 Identify Frame Type T (Normal/Refresh) and select active RLS state $\hat{\theta}_{curr}$

 Note: Prediction uses θ_{ema} , NOT the instantaneous $\hat{\theta}_{curr}$

$QP_{raw} \leftarrow \theta_{ema}[0] \cdot \ln(R_{pic}) + \theta_{ema}[1]$

// Step D: Smart Output Smoothing

$e_{ratio} \leftarrow \text{mean}(E)/R_{tgt}$

if $e_{ratio} < \delta$ **then** (Stability Mode)

$QP_{smooth} \leftarrow \gamma \cdot QP_{t-1} + (1 - \gamma) \cdot QP_{raw}$

else (Emergency Response)

$QP_{smooth} \leftarrow QP_{raw}$

end if

$QP_t \leftarrow \text{clip}(\text{round}(QP_{smooth}), QP_{min}, QP_{max})$

// Step E: State & Budget Update

 Encode frame $\rightarrow$ get R_{real}

$R_{cum} \leftarrow R_{cum} + R_{real}; \quad$ Update queue E

$R_{rem} \leftarrow R_{rem} - R_{real}; \quad \Sigma\omega_{rem} \leftarrow \Sigma\omega_{rem} - \omega_t$

RLS Update: Update $\hat{\theta}_{curr}$ and $\mathbf{P}$ using R_{real} and QP_t

Param Smoothing: $\theta_{ema} \leftarrow \eta \cdot \theta_{ema} + (1 - \eta) \cdot \hat{\theta}_{curr}$

end for

Chapter 5

Experiments and Results



5.1 Experimental Setup

To comprehensively evaluate the effectiveness of the proposed rate control scheme, we conducted extensive experiments on standard video compression benchmarks. This section details the datasets used, the implementation specifics of our framework and baselines, and the evaluation metrics employed.

5.1.1 Datasets

We selected three widely used datasets to evaluate the performance of our method across diverse video contents and motion characteristics. To ensure a fair and consistent comparison, all video sequences were standardized to a resolution of 1920×1080 . Furthermore, we adopted a “Low-Delay P” configuration (intra period set to -1), where only the first frame is encoded as an I-frame, and all subsequent frames are encoded as P-frames. We utilized the full length of each video sequence without cropping to test the long-term stability of the rate control algorithm.

- **UVG Dataset** [61]: This dataset offers high frame rate (120fps) sequences in both 4K and 1080p resolutions. For our experiments, we utilized the native 1080p sequences provided by the dataset. The UVG dataset is characterized by high temporal redundancy and smooth motion, making it suitable for evaluating the efficiency of inter-frame prediction and rate stability over long durations. Most sequences in this

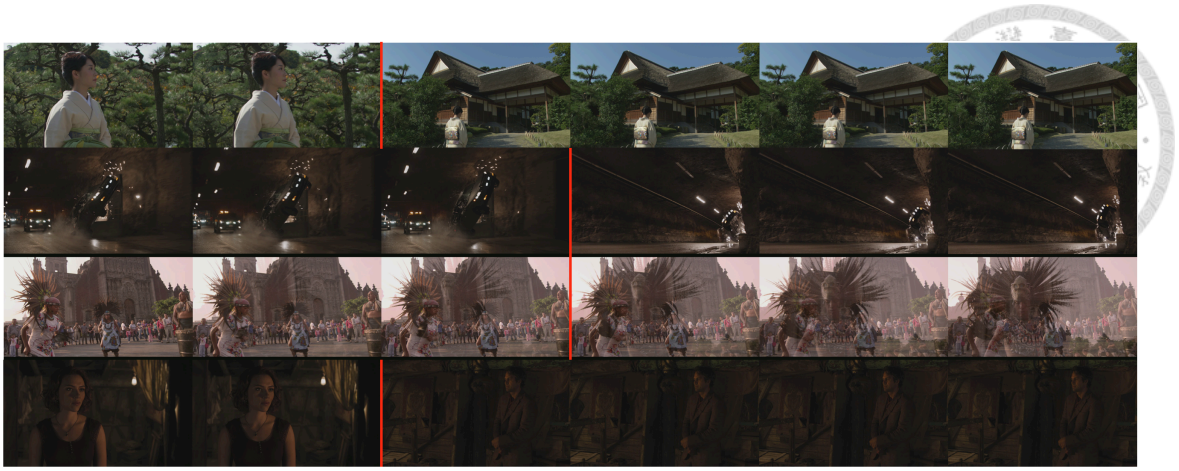
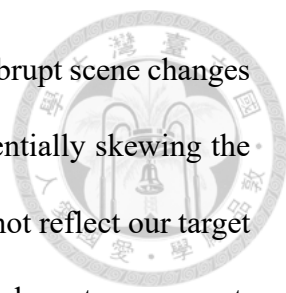


Figure 5.1: Examples of sequences containing hard cuts (editorial shot transitions). To align with the continuous scene characteristic of real-time streaming, sequences such as *Kimono* from HEVC Class B and several clips from MCL-JCV were excluded from our evaluation set.

dataset consist of 600 frames.

- **MCL-JCV Dataset** [62]: The MCL-JCV dataset comprises 30 video sequences with a resolution of 1920×1080 . It covers a wide variety of content types, including distinct textures and motion patterns (e.g., camera panning, zooming, and complex object motion). This diversity is crucial for assessing the generalization capability of the rate control model. The sequence lengths typically range between 120 and 150 frames.
- **HEVC Class B Dataset** [3]: We utilized the standard Class B sequences from the HEVC common test conditions. These sequences are natively 1920×1080 and are widely used in the literature [10, 12] for benchmarking video compression performance. They present challenging scenarios with varying degrees of object motion and background complexity, with lengths ranging from 200 to 600 frames.

Since our target scenario is real-time streaming and video conferencing, which typically feature continuous scenes, we applied a specific filtering criterion to the selected datasets. As illustrated in Fig. 5.1, certain sequences in the HEVC Class B and MCL-



JCV datasets contain editorial shot transitions or “hard cuts.” These abrupt scene changes can cause instantaneous spikes in bit rate or quality fluctuation, potentially skewing the evaluation of the rate control algorithm’s stability in a way that does not reflect our target use case. Therefore, we excluded these sequences to ensure a more relevant assessment. Specifically, the *Kimono* sequence was excluded from the HEVC Class B dataset, and sequences indices [04, 06, 12, 18, 19, 20, 21, 27, 29, 30] were excluded from the MCL-JCV dataset.

5.2 Evaluation Benchmarks

To provide a holistic evaluation, we employed four key metrics focusing on accuracy, quality, stability, and efficiency.

5.2.1 Rate Control Accuracy

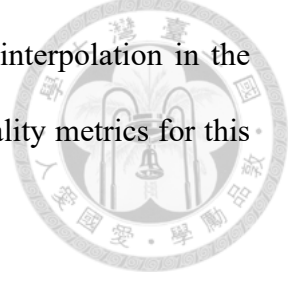
The primary goal of rate control is to minimize the deviation between the target bitrate and the actual output bitrate. We measure this using the absolute Bitrate Error (BRE), defined as:

$$\Delta R = \left| \frac{R_{target} - R_{actual}}{R_{target}} \right| \times 100\% \quad (5.1)$$

5.2.2 R-D Performance

We evaluate the coding efficiency using the Bjøntegaard Delta Rate (BD-Rate) [63]. BD-Rate measures the percentage of bitrate savings required to achieve the same objective quality compared to an anchor. In our implementation, the BD-Rate is calculated

using Piecewise Cubic Hermite Interpolating Polynomial (PCHIP) interpolation in the logarithmic rate domain to ensure curve stability. We utilize two quality metrics for this calculation:



- **PSNR (Peak Signal-to-Noise Ratio):** Evaluating pixel-level fidelity.
- **MS-SSIM (Multi-Scale Structural Similarity):** Evaluating perceptual quality.

Since raw MS-SSIM values often saturate near 1.0, making direct linear interpolation prone to errors, we transform the MS-SSIM values into the decibel (dB) domain prior to calculating the BD-Rate. This transformation is defined as:

$$MS-SSIM_{dB} = -10 \log_{10}(1 - MS-SSIM) \quad (5.2)$$

This logarithmic scaling aligns the metric's behavior with PSNR, allowing for more consistent R-D curve fitting and comparison.

5.2.3 Time Overhead

We measure the computational complexity using the Time Ratio (TR), defined as the total encoding time of the method with rate control divided by the encoding time of the anchor (without rate control):

$$TR = \frac{T_{RC}}{T_{Anchor}} \quad (5.3)$$

A TR close to 1.0 indicates negligible computational overhead.



5.3 Implementation Details

5.3.1 Experimental Settings

To ensure a comprehensive evaluation of the rate control algorithms, we configured the DCVC-RT framework with specific encoding parameters (summarized in Fig. 5.2).

- **Target Rates:** We selected four operating points to represent a wide range of quality levels. The target bitrates (R_{target}) were generated using the original DCVC-RT model with base QP values of $\{5, 20, 35, 50\}$.
- **Intra Period:** We adopted a “Low-Delay P” configuration by setting the Intra Period to -1 . This implies that only the very first frame of the sequence is encoded as an I-frame, while all subsequent frames are encoded as P-frames, simulating a real-time streaming scenario.
- **Sequence Length:** Unlike many studies that test only short clips, we utilized the **full length** of each video sequence (ranging from 100 to 600 frames depending on the dataset) without cropping. This allows us to rigorously test the long-term stability and error propagation of the rate control methods.

5.3.2 Baseline Methods

We compared our proposed method against several state-of-the-art baselines to validate its performance. The baselines include:

- **Anchor (Rule-based):** The rule-based rate control method proposed by the DCVC authors in the Appendix of DCVC-FM [53]. It incorporates the framework’s default

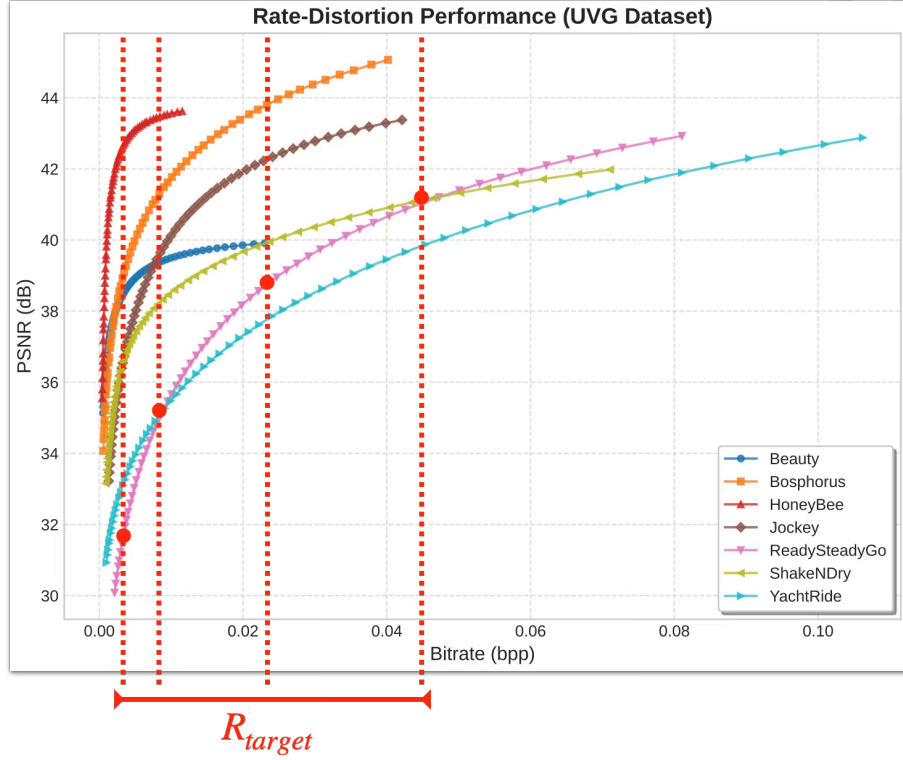


Figure 5.2: Rate-Distortion performance on the UVG dataset. The red dotted lines illustrate our target rate selection strategy: four distinct bitrates are determined based on the anchor QP settings (5, 20, 35, 50) to evaluate the controller’s performance across different bandwidth constraints.

inter-frame QP variation patterns to determine the bit budget.

- **Baseline-Lambda (2014)** [7]: A representative λ -domain rate control mechanism. Although originally designed for traditional codecs, the underlying $R - \lambda$ relationship remains applicable to modern codecs.
- **Baseline 2022p (Li et al., ICASSP 2022)** [10]: A pioneering rate control scheme for LVC that proposes an R-D- Λ model to characterize inter-frame dependency. It employs a staged parameter update mechanism to ensure robust estimation and accurate rate allocation.
- **Baseline 2024 (Liao et al., ICASSP 2024)** [11]: A generalizable plug-in rate control scheme that introduces a Rate-Quality ($R - Q_s$) model. Unlike traditional methods, it regulates the bitrate by directly adjusting the scaling parameter Q_s to preserve

the hierarchical quality structure of NVC frameworks.



For our proposed methods, we evaluate two variations:

- **Proposed-Iter:** Uses the QP-log(R) model with the iterative gradient descent update method.
- **Proposed-RLS:** Utilizes the Recursive Least Squares (RLS) algorithm for parameter updates, which is expected to yield superior stability and adaptability.

5.3.3 Parameter Settings

We implemented the three baseline methods and made necessary adjustments to adapt them to DCVC-RT. Through extensive parameter search experiments, we selected the optimal settings for each method.

For **Baseline-Lambda (2014)** [7], we adopted the $R - \lambda$ model initialized via fitting on the Bosphorus sequence. The detailed parameter configurations are listed in Table 5.1.

Table 5.1: Parameter settings for Baseline-Lambda.

Category	Parameter	Value	Description
Model	Initial α (α_0)	10.0	Scale factor from fitting
	Initial β (β_0)	1.76	Shape factor from fitting
	Learning Rate α	0.1	Update step size
	Learning Rate β	0.05	Update step size
Control	Max Ratio (r_{up})	1.30	Max λ increase (+30%)
	Min Ratio (r_{down})	0.90	Max λ decrease (-10%)
	GOP Size	8	Rebalancing interval
	Sliding Window	40	Smoothing window size

For **Baseline 2022p (Li et al., 2022)** [10], which extends the λ -domain model with an inter-frame dependency factor p , the specific configuration is provided in Table 5.2.

Table 5.2: Parameter settings for Baseline 2022p.

Category	Parameter	Value	Description
Model	Initial α (α_0)	10.0	Base scale factor
	Initial β (β_0)	1.76	Base shape factor
	Initial p (p_0)	0.2	Dependency factor
	Parameter k	0.1	Update coefficient
	p Range	[1e-6, 0.5]	Constraint for stability
Control	Max Ratio (r_{up})	1.30	Max λ increase
	Min Ratio (r_{down})	0.90	Max λ decrease

For **Baseline 2024 (Liao et al., 2024)** [11], we implemented the $R - Q_s$ model. To align with the positive correlation between QP and bitrate in DCVC-RT, we adapted the model parameters as detailed in Table 5.3.

Table 5.3: Parameter settings for Baseline 2024.

Category	Parameter	Value	Description
Model	Initial C (C_{init})	0.1	Adapted scale factor
	Initial K (K_{init})	2.0	Power factor
	LR for C (η)	0.01	Update step size
	LR for K (μ)	0.005	Update step size
Allocation	GOP Size	32	Rebalancing interval
	Sliding Window	40	Smoothing window size

For the proposed methods, we evaluate two variations. Table 5.4 details the configuration for the **Proposed-Iter** method, which utilizes a gradient descent approach on the $QP - \log(R)$ model.

Table 5.4: Parameter settings for Proposed-Iter.

Category	Parameter	Value	Description
Model	Initial a (a_0)	14.5	Slope coefficient
	Initial b (b_0)	110.0	Intercept coefficient
	LR for a (η_a)	0.01	Gradient step size
	LR for b (η_b)	0.10	Gradient step size
Allocation	GOP Size	16	Rebalancing interval
	Sliding Window	40	Smoothing window size
	Floor Ratio	0.2	Min bit budget protection

Table 5.5 presents the settings for the **Proposed-RLS** method. This version incor-

porates Recursive Least Squares for parameter estimation and a Smart Smoothing mechanism to enhance stability.



Table 5.5: Parameter settings for Proposed-RLS.

Category	Parameter	Value	Description
Model	Initial a (a_0)	14.5	Base slope
	Initial b (b_0)	110.0	Base intercept
RLS Control	Forgetting Factor (λ)	0.995	Memory length for RLS
	EMA Factor (α)	0.8	Short-term trend smoothing
	Smooth Factor	0.8	QP variation damping
	Safety Margin	0.1	Error tolerance threshold
Allocation	GOP Size	8	Tighter rebalancing interval
	Sliding Window	40	Smoothing window size
	Floor Ratio	0.2	Increased protection ratio

5.4 Performance Comparison

5.4.1 Quantitative Results on UVG Dataset

Table 5.6 presents the comparative results on the UVG dataset. The “Anchor” refers to the default rule-based method in DCVC-RT.

First, we analyze the applicability of the baselines. As observed in Table 5.6, **Baseline-2022p (Li et al., 2022)** and **Baseline-2024 (Liao et al., 2024)** exhibit significantly high Bitrate Error (BRE) and unstable R-D performance. Baseline-2022p relies on an inter-frame dependency factor p . In our “Low-Delay P” configuration (Intra Period = -1), the absence of periodic I-frame resets causes estimation errors to accumulate, leading to severe error propagation. Similarly, Baseline-2024 employs an $R - Q_s$ model tailored for a different architecture. Despite grid search optimization, the model fails to align with the rate-distortion characteristics of DCVC-RT, resulting in model mismatch.

Consequently, these two baselines are excluded from subsequent detailed discus-

sions. The comparison focuses on the **Anchor** and the **Baseline-Lambda (2014)**, which generalizes better across architectures.

Table 5.6: Comparison of Rate Control Performance on UVG Dataset. Metrics include Bitrate Error (BRE %) and BD-Rate (PSNR). **Bold** indicates the best performance, and underline indicates the second best. (Note: **Lower** BRE indicates better accuracy; **more negative** BD-Rate indicates better coding efficiency.)

Sequence	Anchor		Baseline-Lambda		Baseline-2022p		Baseline-2024		Prop-Iter		Prop-RLS	
	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R
Beauty	2.48	0.0	0.73	2.4	122.17	<u>-2.5</u>	47.84	9.2	0.10	-1.6	<u>0.15</u>	-4.0
Bosphorus	3.64	0.0	<u>2.47</u>	3.9	248.19	-0.7	50.54	-8.4	0.28	-17.6	4.09	<u>-15.3</u>
HoneyBee	<u>4.38</u>	0.0	4.89	<u>-43.9</u>	79.12	-38.4	58.09	-14.8	7.53	-41.4	1.85	-49.5
Jockey	1.62	0.0	<u>0.18</u>	-2.4	228.62	-4.2	28.43	3.8	0.06	-0.5	3.21	<u>-3.4</u>
ReadySteadyGo	2.47	0.0	1.19	-3.5	356.04	-3.8	21.47	-2.5	0.28	<u>-4.4</u>	0.96	-6.1
ShakeNDry	<u>3.32</u>	0.0	7.60	-10.7	347.81	2.5	35.98	<u>-16.3</u>	8.76	-14.9	2.92	-21.4
YachtRide	2.28	0.0	1.67	-1.0	598.36	-0.9	22.21	0.2	0.14	<u>-2.9</u>	0.24	-4.7
Average	2.88	0.0	2.68	-7.9	282.90	-6.9	37.79	-4.1	<u>2.45</u>	<u>-11.9</u>	1.92	-14.9

The results demonstrate that our proposed RLS-based method significantly outperforms the baselines. While the Anchor and Baseline-Lambda maintain reasonable bitrate accuracy (Average BRE of 2.88% and 2.68%, respectively), the **Proposed-RLS** achieves a superior Average BRE of **1.92%**. More importantly, in terms of coding efficiency, the proposed method achieves an average BD-Rate reduction of **14.9%**, significantly surpassing the 7.9% reduction achieved by Baseline-Lambda.

5.4.2 Generalization on Diverse Datasets

We further extended the evaluation to the MCL-JCV and HEVC Class B datasets to assess the generalization capability of our method. Table 5.7 and Table 5.8 summarize the results.

Discussion and Insights

The experimental results across the UVG, HEVC-B, and MCL-JCV datasets reveal a distinct correlation between the temporal stability of the video content and the efficacy



Table 5.7: Rate Control Performance on the MCL-JCV Dataset. **Bold** indicates best, underline indicates second best. (Note: **Lower** BRE is better; **more negative** BD-Rate is better.)

Sequence (videoSRC)	Anchor		Baseline-Lambda		Prop-RLS	
	BRE (%)	BD-Rate	BRE (%)	BD-Rate	BRE (%)	BD-Rate
01	12.19	0.0	5.60	<u>-37.9</u>	<u>9.07</u>	-48.3
02	<u>3.44</u>	0.0	0.55	<u>-5.5</u>	5.68	-9.6
03	10.89	0.0	<u>3.71</u>	<u>-21.9</u>	1.21	-27.5
05	6.40	<u>0.0</u>	0.13	-5.4	<u>1.28</u>	3.2
07	8.11	<u>0.0</u>	0.27	<u>0.0</u>	<u>1.63</u>	-8.4
08	10.63	<u>0.0</u>	1.00	7.8	<u>2.02</u>	-19.9
09	11.26	0.0	0.92	-8.2	<u>3.64</u>	<u>-6.3</u>
10	2.29	<u>0.0</u>	<u>0.29</u>	-3.8	0.22	7.5
11	5.75	0.0	0.51	-6.0	<u>1.47</u>	<u>-5.2</u>
13	<u>17.83</u>	<u>0.0</u>	20.57	-16.4	8.26	2.1
14	6.72	<u>0.0</u>	0.66	2.9	<u>2.57</u>	-3.1
15	10.65	0.0	<u>8.92</u>	<u>-43.5</u>	2.75	-45.5
16	11.86	0.0	<u>1.17</u>	<u>-12.5</u>	0.13	-14.9
17	9.47	0.0	0.36	<u>-3.3</u>	<u>0.69</u>	-4.7
22	4.79	0.0	0.31	-14.7	<u>2.48</u>	<u>-9.3</u>
23	9.62	0.0	0.67	<u>-10.3</u>	<u>2.57</u>	-12.6
24	6.35	0.0	<u>0.84</u>	<u>-0.3</u>	0.30	-4.9
25	7.56	<u>0.0</u>	<u>3.54</u>	-2.0	3.51	2.6
26	5.58	<u>0.0</u>	1.25	3.1	<u>2.15</u>	-3.1
28	10.94	0.0	2.77	<u>-22.5</u>	<u>4.50</u>	-24.9
Average	8.62	0.0	2.70	<u>-10.02</u>	<u>2.81</u>	-11.64

Table 5.8: Rate Control Performance on the HEVC Class B Dataset. (Note: **Lower** BRE is better; **more negative** BD-Rate is better.)

Sequence	Anchor		Baseline-Lambda		Prop-RLS	
	BRE (%)	BD-Rate	BRE (%)	BD-Rate	BRE (%)	BD-Rate
BQTerrace	6.63	0.0	3.82	3.7	<u>4.17</u>	-18.6
BasketballDrive	1.64	0.0	<u>0.23</u>	<u>-2.6</u>	0.15	-5.9
Cactus	3.95	0.0	<u>1.34</u>	<u>-7.0</u>	0.19	-10.0
ParkScene	7.90	0.0	2.85	<u>-22.7</u>	<u>7.89</u>	-24.3
Average	5.03	0.0	2.06	<u>-7.15</u>	<u>3.10</u>	-14.70

of the proposed RLS-based control.

As evidenced by the results, the **Prop-RLS** method yields the most significant coding efficiency gains on high-frame-rate datasets with stable motion. On the UVG dataset (120 fps), which features smooth camera movements and high temporal redundancy, our method achieves a substantial BD-Rate reduction of **14.9%**. Similarly, on the HEVC Class B dataset (typically 50–60 fps), sequences like *BasketballDrive* and *Cactus* exhibit strong performance improvements. This can be attributed to the RLS algorithm’s ability to effectively leverage historical data to refine its parameter estimation when inter-frame correlations are strong and consistent.

Conversely, for content with lower frame rates or more erratic motion, the advantage of the RLS approach becomes less pronounced. For instance, in the *ParkScene* sequence (24 fps), while the RLS method still outperforms the Baseline-Lambda in terms of BD-Rate (-24.3% vs. -22.7%), the margin is narrower compared to the 60 fps sequences, and the bitrate error (BRE) slightly increases. In such complex, fast-changing scenarios, the historical data becomes less predictive of future frames, causing the performance to converge towards that of a standard gradient-based controller.

In conclusion, our proposed method aligns well with the target scenario of **video conferencing and real-time streaming**, which typically involves fixed-camera setups, stable viewpoints, and continuous scenes. Under these conditions, the RLS-based rate control successfully maximizes coding efficiency while maintaining stable bitrate regulation.

5.4.3 Environment Specifications and Computational Overhead

Finally, we evaluated the time complexity of the implemented rate control schemes. Experiments were conducted on a single NVIDIA GeForce RTX 5090 GPU. All methods were executed in a single-process mode and tested on the "Beauty" sequence (600 frames) from the UVG dataset. The tests were repeated four times, yielding an average inference time of approximately 7 ms per frame. To assess the computational cost, we calculated the Time Ratio (TR) using the original anchor model (sourced from the official GitHub repository without rate control) as the baseline. As shown in the summary below, all methods exhibit a TR close to 1:

Table 5.9: Comparison of Time Overhead relative to the Rule-Based method.

	Rule-Based	2014-lambda	Prop-RLS
Time Overhead	1.003	1.003	1.013

These results confirm that the proposed algorithm, along with the baselines, operates with negligible computational overhead. They are fully capable of supporting one-pass rate control in real-time systems without bottlenecking the encoding process.

5.5 Parametric Analysis and Robustness

A key advantage of the proposed RLS-based controller is its robustness against hyperparameter initialization and tuning. Traditional gradient-based methods are typically sensitive to initial values and step sizes (learning rates), often requiring precise tuning to avoid divergence. To validate the superiority of our method, we conducted extensive ablation studies focusing on parameter initialization and update dynamics.



5.5.1 Robustness to Initialization

We first examined the impact of the initial model parameters on Rate-Distortion (R-D) performance. Table 5.10 and Table 5.11 detail the performance of the **Proposed-RLS** and the **Baseline-Lambda** methods under various initialization settings.

For the **Proposed-RLS** method (Table 5.10), the parameter a represents the slope in the logarithmic domain ($QP \approx a \cdot \log(R) + b$). We tested initial values ranging from $a = 12$ to $a = 20$. The results demonstrate remarkable stability. The best performance is achieved at **Init-A16** (Average BD-Rate of -14.9%). Even when the initialization deviates significantly—for instance, at **Init-A12** or **Init-A20**—the method continues to provide substantial coding gains (Average BD-Rates of -13.1% and -6.5%, respectively). This indicates that the RLS algorithm can effectively correct initial estimation errors via its covariance matrix updates, preventing performance collapse.

Table 5.10: Ablation Study on Initialization Parameter (Proposed-RLS). **Bold** indicates the best performance. (Note: Robustness is shown by maintaining low BRE and negative BD-Rate across settings.)

Sequence	Init-A12		Init-A14		Init-A16		Init-A18		Init-A20	
	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R
Beauty	0.17	-4.1	0.19	-5.0	0.15	-4.0	0.15	-3.2	0.18	-2.6
Bosphorus	7.50	-13.9	7.56	-15.4	4.09	-15.3	12.97	-13.4	7.92	-9.3
HoneyBee	17.17	-41.1	2.97	-48.8	1.85	-49.5	2.73	-41.8	4.14	-42.6
Jockey	3.24	-2.5	3.22	-2.8	3.21	-3.4	3.30	-3.6	3.39	-3.0
ReadySteadyGo	0.59	-5.1	0.56	-5.7	0.96	-6.1	0.86	-5.3	0.57	-2.6
ShakeNDry	4.74	-21.0	10.97	-18.5	2.92	-21.4	4.95	31.6	4.26	20.5
YachtRide	0.15	-4.0	0.20	-4.5	0.24	-4.7	0.36	-5.8	0.25	-5.8
Average	4.79	-13.1	3.67	-14.4	1.92	-14.9	3.62	-5.9	2.96	-6.5

In stark contrast, the **Baseline-Lambda** (Table 5.11) exhibits high sensitivity to its scaling parameter α . While it performs well at the optimal setting (**Init-A10**, BD-Rate -7.9%), the performance degrades drastically as α increases. At **Init-A500** and **Init-A1000**, the method fails to converge properly, resulting in a catastrophic increase in BD-Rate

(+10.1% and +16.3% respectively), effectively acting as a coding loss. This comparison highlights that the RLS-based approach is significantly more robust to initial parameter deviations.

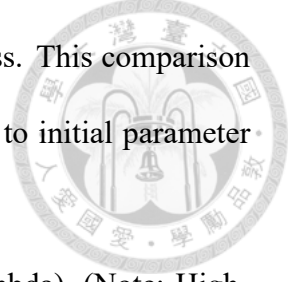


Table 5.11: Ablation Study on Initialization Parameter (Baseline-Lambda). (Note: High BRE and positive BD-Rate indicate model divergence.)

Sequence	Init-A10		Init-A50		Init-A100		Init-A500		Init-A1000	
	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R
Beauty	0.73	2.4	0.62	0.7	0.99	-1.4	0.81	6.6	1.45	12.1
Bosphorus	2.47	3.9	4.72	22.1	3.83	32.1	7.06	67.8	8.32	73.9
HoneyBee	4.89	-43.9	6.67	-34.9	5.86	-37.3	7.68	-27.6	11.57	-16.0
Jockey	0.18	-2.4	0.24	-1.8	0.43	-1.7	0.73	-1.3	0.73	0.6
ReadySteadyGo	1.19	-3.5	1.35	-1.8	1.72	1.1	2.12	14.9	2.05	28.2
ShakeNDry	7.60	-10.7	3.93	-19.8	4.07	-22.8	6.16	-9.9	5.04	-11.3
YachtRide	1.67	-1.0	1.23	-2.8	2.76	2.9	2.59	20.3	2.78	26.3
Average	2.68	-7.9	2.68	-5.5	2.81	-3.9	3.88	10.1	4.56	16.3

5.5.2 Sensitivity to Update Dynamics

We further analyzed the sensitivity of the update mechanisms: the Forgetting Factor (λ) for the Proposed-RLS method and the Learning Rate (LR) for the Baseline-Lambda.

Table 5.12 presents the performance of Proposed-RLS with varying λ . The method exhibits high stability across a wide range of values. From $\lambda = 0.960$ to $\lambda = 0.995$, the Average BD-Rate remains consistently negative (better than -10.9%), with the optimal performance observed at **Lambda-995** (BD-Rate -14.9%). This suggests that the RLS formulation is not brittle and can maintain effective control without requiring extremely precise tuning of the forgetting factor.

On the other hand, Table 5.13 reveals that the Baseline-Lambda is highly sensitive to the Learning Rate. While moderate rates (e.g., Lr0.05 and Lr0.1) yield good performance, increasing the learning rate to **Lr0.5** causes the model to diverge, resulting in a massive

Table 5.12: Ablation Study on Forgetting Factor λ (Proposed-RLS). **Bold** indicates the best performance. (Note: Consistently negative BD-Rate indicates high stability.)

Sequence	Lambda-960		Lambda-970		Lambda-980		Lambda-990		Lambda-995	
	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R
Beauty	0.09	-3.7	0.13	-3.8	0.16	-4.1	0.18	-4.1	0.15	-4.0
Bosphorus	8.58	-5.4	7.41	-6.2	3.59	-8.6	2.58	-11.5	4.09	-15.3
HoneyBee	0.87	-49.6	1.78	-49.8	2.89	-34.8	1.99	-49.0	1.85	-49.5
Jockey	0.90	1.2	1.24	1.5	1.84	1.5	4.35	-2.3	3.21	-3.4
ReadySteadyGo	1.99	-7.8	1.79	-7.4	1.62	-7.5	1.40	-7.1	0.96	-6.1
ShakeNDry	5.02	-20.0	18.05	-12.0	5.33	-18.0	3.12	-20.5	2.92	-21.4
YachtRide	0.07	-4.8	0.01	-4.8	0.05	-4.8	0.05	-4.8	0.24	-4.7
Average	2.50	-12.9	4.34	-11.8	2.21	-10.9	1.95	-14.2	1.92	-14.9

performance penalty (Average BD-Rate of +34.3%). This confirms that the proposed RLS method offers superior stability and ease of tuning compared to traditional gradient-based approaches.

Table 5.13: Ablation Study on Learning Rate (Baseline-Lambda). **Bold** indicates the best performance. (Note: Large positive BD-Rate values at Lr0.5 indicate catastrophic failure.)

Sequence	Lr0.025		Lr0.05		Lr0.1		Lr0.25		Lr0.5	
	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R	BRE	BD-R
Beauty	3.49	5.7	0.32	5.3	0.73	2.4	0.78	9.7	2.29	40.3
Bosphorus	4.52	-9.8	1.27	-5.0	2.47	3.9	5.84	49.3	7.12	73.2
HoneyBee	2.84	-25.2	3.81	-32.2	4.89	-43.9	18.54	14.1	23.02	51.5
Jockey	2.64	-2.2	1.11	-2.8	0.18	-2.4	0.86	0.0	2.12	17.8
ReadySteadyGo	2.52	-4.6	1.22	-3.9	1.19	-3.5	1.35	-1.2	2.64	25.6
ShakeNDry	10.30	-4.2	9.92	6.9	7.60	-10.7	6.08	-12.4	6.96	13.4
YachtRide	2.53	-2.1	0.88	-1.0	1.67	-1.0	1.49	0.4	2.78	18.3
Average	4.12	-6.1	2.65	-4.7	2.68	-7.9	4.99	8.6	6.70	34.3

5.6 Summary

In this chapter, we presented a comprehensive evaluation of the proposed rate control framework within the DCVC-RT architecture. Through extensive experiments across the UVG, MCL-JCV, and HEVC Class B datasets, we validated the superiority of the RLS-based approach compared to both rule-based and gradient-based baselines.

The key findings from our experiments can be summarized as follows:

- **Superior Coding Efficiency:** The Proposed-RLS method consistently achieved the best Rate-Distortion performance, yielding an average BD-Rate reduction of 14.9% on the UVG dataset. The insights reveal that the method is particularly effective for high-frame-rate sequences and continuous scenes typical of video conferencing.
- **High Accuracy and Stability:** With an average Bitrate Error (BRE) of 1.92%, the proposed controller demonstrated precise rate regulation capabilities, effectively minimizing deviations from the target bandwidth.
- **Robustness:** Ablation studies confirmed that the RLS algorithm exhibits remarkable robustness against initial parameter mismatches and is insensitive to hyperparameter variations. This contrasts sharply with traditional gradient-based methods, which are prone to divergence under suboptimal settings.
- **Real-Time Feasibility:** The computational complexity analysis showed that our method introduces negligible overhead (Time Ratio ≈ 1.01), ensuring that the real-time performance of DCVC-RT is preserved.

Collectively, these results demonstrate that the proposed RLS-based rate control scheme provides a robust, efficient, and high-performance solution tailored for real-time neural video coding applications.



Chapter 6

Conclusion



6.1 Summary of Contributions

This thesis has presented a robust rate control framework tailored for real-time Neural Video Coding (NVC), specifically DCVC-RT. Addressing the challenges of conditional coding and periodic refresh mechanisms, we proposed an Adaptive State-Space Rate Control system. The primary contributions are summarized as follows:

1. **Dual-State RLS Estimation:** We established a Log-linear R-QP model and proposed a **Dual-State Recursive Least Squares (RLS)** algorithm. By temporally decoupling the estimation for standard P-frames and Refresh frames, this method resolves the “Model Shattering” phenomenon, preventing parameter corruption caused by systemic outliers.
2. **Stabilized Control Loop:** We introduced a hierarchical budgeting strategy combined with a **Double-Smoothing mechanism** (Parameter EMA and Smart Output Smoothing). This design effectively filters estimation noise while dynamically balancing buffer safety against visual stability.

Experimental results on the UVG dataset demonstrate that our framework achieves an average BD-Rate reduction of **14.9%** compared to the anchor. Furthermore, the ablation studies confirm that the proposed RLS-based method exhibits superior robustness to hyperparameter variations compared to traditional gradient-based approaches.

6.2 Limitations and Future Work



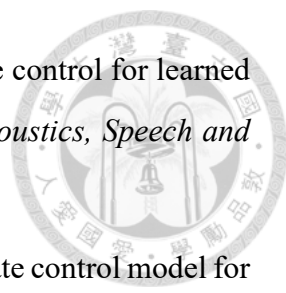
While the proposed method demonstrates high efficiency and stability, there are promising avenues for future research:

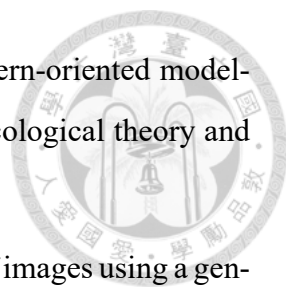
- **Scene Cut Adaptation:** The current RLS estimator relies on historical statistics, which can be disrupted by abrupt shot changes. Future work should incorporate a scene-change detector to reset the covariance matrix, enabling faster convergence during hard cuts.
- **End-to-End Joint Optimization:** This study treats the encoder as a black-box model. A potential direction is to jointly train a lightweight rate control network with the compression backbone, allowing the system to leverage internal latent features for more precise "look-ahead" bit allocation.

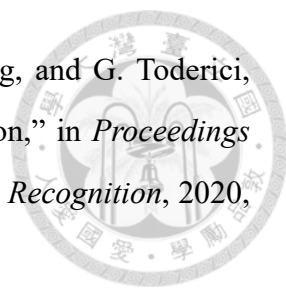
References

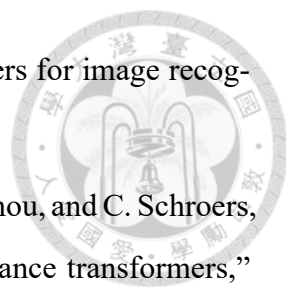


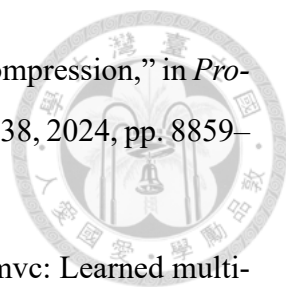
- [1] Ericsson, “Ericsson Mobility Report,” Ericsson, Stockholm, Sweden, Tech. Rep., Jun. 2025, June 2025 edition. Accessed: Nov. 9, 2025. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/mobility-report>.
- [2] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, “Overview of the h. 264/avc video coding standard,” *IEEE Transactions on circuits and systems for video technology*, vol. 13, no. 7, pp. 560–576, 2003.
- [3] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, “Overview of the high efficiency video coding (hevc) standard,” *IEEE Transactions on circuits and systems for video technology*, vol. 22, no. 12, pp. 1649–1668, 2012.
- [4] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, “Overview of the versatile video coding (vvc) standard and its applications,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 10, pp. 3736–3764, 2021.
- [5] D. Liu, Y. Li, J. Lin, H. Li, and F. Wu, “Deep learning-based video coding: A review and a case study,” *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–35, 2020.
- [6] J. Ascenso, E. Alshina, and T. Ebrahimi, “The jpeg ai standard: Providing efficient human and machine visual data consumption,” *Ieee Multimedia*, vol. 30, no. 1, pp. 100–111, 2023.
- [7] B. Li, H. Li, L. Li, and J. Zhang, “ λ domain rate control algorithm for High Efficiency Video Coding,” *IEEE Transactions on Image Processing*, vol. 23, no. 9, pp. 3841–3854, 2014.
- [8] Z. Jia, B. Li, J. Li, W. Xie, L. Qi, H. Li, and Y. Lu, “Towards practical real-time neural video compression,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 12 543–12 552.
- [9] *Enhanced compression model (ecm) reference software*, <https://vcgit.hhi.fraunhofer.de/ecm/>, JVET experimental reference software.

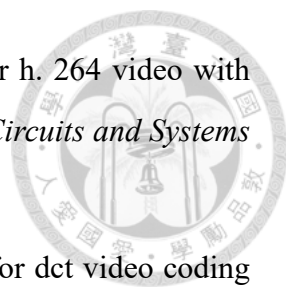
- 
- [10] Y. Li, X. Chen, J. Li, J. Wen, Y. Han, S. Liu, and X. Xu, “Rate control for learned video compression,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 2829–2833.
 - [11] S. Liao, C. Jia, H. Fan, J. Yan, and S. Ma, “Rate-quality based rate control model for neural video compression,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2024, pp. 4215–4219.
 - [12] Y. Zhang, G. Lu, Y. Chen, S. Wang, Y. Shi, J. Wang, and L. Song, “Neural rate control for learned video compression,” in *The Twelfth International Conference on Learning Representations*, 2024.
 - [13] S. S. Haykin, *Adaptive filter theory*. Pearson Education India, 2002.
 - [14] C. E. Shannon, “A mathematical theory of communication,” *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
 - [15] K. Sayood, *Introduction to data compression*. Morgan Kaufmann, 2017.
 - [16] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, “Video enhancement with task-oriented flow,” *International Journal of Computer Vision*, vol. 127, no. 8, pp. 1106–1125, 2019.
 - [17] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 2007.
 - [18] G. Lu, W. Ouyang, D. Xu, X. Zhang, C. Cai, and Z. Gao, “Dvc: An end-to-end deep video compression framework,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 11 006–11 015.
 - [19] K. O’shea and R. Nash, “An introduction to convolutional neural networks,” *arXiv preprint arXiv:1511.08458*, 2015.
 - [20] A. Ranjan and M. J. Black, “Optical flow estimation using a spatial pyramid network,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4161–4170.

- 
- [21] T. Wiegand, F. Jeltsch, I. Hanski, and V. Grimm, “Using pattern-oriented modeling for revealing hidden information: A key for reconciling ecological theory and application,” *Oikos*, vol. 100, no. 2, pp. 209–222, 2003.
 - [22] J. Ballé, V. Laparra, and E. P. Simoncelli, “Density modeling of images using a generalized normalization transformation,” *arXiv preprint arXiv:1511.06281*, 2015.
 - [23] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, “Variational image compression with a scale hyperprior,” *arXiv preprint arXiv:1802.01436*, 2018.
 - [24] J. Bégaint, F. Racapé, S. Feltman, and A. Pushparaja, “Compressai: A pytorch library and evaluation platform for end-to-end compression research,” *arXiv preprint arXiv:2011.03029*, 2020.
 - [25] J. Ballé, V. Laparra, and E. P. Simoncelli, “End-to-end optimized image compression,” *arXiv preprint arXiv:1611.01704*, 2016.
 - [26] D. Marpe, H. Schwarz, and T. Wiegand, “Context-based adaptive binary arithmetic coding in the h.264/avc video compression standard,” *IEEE Transactions on circuits and systems for video technology*, vol. 13, no. 7, pp. 620–636, 2003.
 - [27] C.-H. Huang and J.-L. Wu, “Unveiling the future of human and machine coding: A survey of end-to-end learned image compression,” *Entropy*, vol. 26, no. 5, p. 357, 2024.
 - [28] D. Minnen, J. Ballé, and G. D. Toderici, “Joint autoregressive and hierarchical priors for learned image compression,” *Advances in neural information processing systems*, vol. 31, 2018.
 - [29] G.-H. Wang, J. Li, B. Li, and Y. Lu, “Evc: Towards real-time neural image compression with mask decay,” *arXiv preprint arXiv:2302.05071*, 2023.
 - [30] G. Lu, X. Zhang, W. Ouyang, L. Chen, Z. Gao, and D. Xu, “An end-to-end learning framework for video compression,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 10, pp. 3292–3308, 2020.

- 
- [31] E. Agustsson, D. Minnen, N. Johnston, J. Balle, S. J. Hwang, and G. Toderici, “Scale-space flow for end-to-end optimized video compression,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8503–8512.
 - [32] Y. Wang, L. Lipson, and J. Deng, “Sea-raft: Simple, efficient, accurate raft for optical flow,” in *European Conference on Computer Vision*, Springer, 2024, pp. 36–54.
 - [33] J. Yang, C. Yang, Y. Zhai, Q. Wang, X. Pan, and R. Wang, “Improving learned video compression by exploring spatial redundancy,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 2860–2864.
 - [34] Y. Shi, Y. Ge, J. Wang, and J. Mao, “Alphavc: High-performance and efficient learned video compression,” in *European Conference on Computer Vision*, Springer, 2022, pp. 616–631.
 - [35] C. Reich, B. Debnath, D. Patel, T. Prangemeier, D. Cremers, and S. Chakradhar, “Deep video codec control for vision models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 5732–5741.
 - [36] Z. Chen, H. Sun, L. Zhang, and F. Zhang, “Survey on visual signal coding and processing with generative models: Technologies, standards, and optimization,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 14, no. 2, pp. 149–171, 2024.
 - [37] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.
 - [38] E. Agustsson, D. Minnen, G. Toderici, and F. Mentzer, “Multi-realism image compression with a conditional generator,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 22 324–22 333.
 - [39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.

- 
- [40] A. Dosovitskiy, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
 - [41] Z. Chen, L. Relic, R. Azevedo, Y. Zhang, M. Gross, D. Xu, L. Zhou, and C. Schroers, “Neural video compression with spatio-temporal cross-covariance transformers,” in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 8543–8551.
 - [42] F. Mentzer, G. Toderici, D. Minnen, S.-J. Hwang, S. Caelles, M. Lucic, and E. Agustsson, “Vct: A video compression transformer,” *arXiv preprint arXiv:2206.07307*, 2022.
 - [43] T. Ladune, P. Philippe, W. Hamidouche, L. Zhang, and O. Déforges, “Optical flow and mode selection for learning-based video coding,” in *2020 IEEE 22nd International Workshop on Multimedia Signal Processing (MMSP)*, IEEE, 2020, pp. 1–6.
 - [44] J. Li, B. Li, and Y. Lu, “Deep contextual video compression,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 18 114–18 125, 2021.
 - [45] X. Sheng, J. Li, B. Li, L. Li, D. Liu, and Y. Lu, “Temporal context mining for learned video compression,” *IEEE Transactions on Multimedia*, vol. 25, pp. 7311–7322, 2022.
 - [46] J. Lin, D. Liu, J. Liang, H. Li, and F. Wu, “A deeply modulated scheme for variable-rate video compression,” in *2021 IEEE International Conference on Image Processing (ICIP)*, IEEE, 2021, pp. 3722–3726.
 - [47] Z. Hu and D. Xu, “Complexity-guided slimmable decoder for efficient deep video compression,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 358–14 367.
 - [48] J. Li, B. Li, and Y. Lu, “Hybrid spatial-temporal entropy modelling for neural video compression,” in *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 1503–1511.

- 
- [49] M. Lu, Z. Duan, F. Zhu, and Z. Ma, “Deep hierarchical video compression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 8859–8867.
- [50] B. Liu, Y. Chen, R. C. Machineni, S. Liu, and H.-S. Kim, “Mmvc: Learned multi-mode video compression with block-based prediction mode selection and density-adaptive entropy coding,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 18 487–18 496.
- [51] J. Li, B. Li, and Y. Lu, “Neural video compression with diverse contexts,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 22 616–22 626.
- [52] L. Qi, J. Li, B. Li, H. Li, and Y. Lu, “Motion information propagation for neural video compression,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6111–6120.
- [53] J. Li, B. Li, and Y. Lu, “Neural video compression with feature modulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 26 099–26 108.
- [54] D. Alexandre, H.-M. Hang, and W.-H. Peng, “Hierarchical b-frame video coding using two-layer canf without motion coding,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 10 249–10 258.
- [55] J. Ballé, N. Johnston, and D. Minnen, “Integer networks for data compression with latent-variable models,” in *International Conference on Learning Representations*, 2018.
- [56] G. Gao, H. M. Kwan, F. Zhang, and D. Bull, “Pnvc: Towards practical inr-based video compression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 3068–3076.
- [57] S. Ma, W. Gao, and Y. Lu, “Rate-distortion analysis for h. 264/avc video coding and its application to rate control,” *IEEE transactions on circuits and systems for video technology*, vol. 15, no. 12, pp. 1533–1544, 2005.

- 
- [58] D.-K. Kwon, M.-Y. Shen, and C.-C. J. Kuo, “Rate control for h. 264 video with enhanced rate and distortion models,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 17, no. 5, pp. 517–529, 2007.
 - [59] Z. He, Y. K. Kim, and S. K. Mitra, “Low-delay rate control for dct video coding via/spl rho/-domain source modeling,” *IEEE transactions on Circuits and Systems for Video Technology*, vol. 11, no. 8, pp. 928–940, 2001.
 - [60] B. Li, J. Xu, D. Zhang, and H. Li, “Qp refinement according to lagrange multiplier for high efficiency video coding,” in *2013 IEEE International Symposium on Circuits and Systems (ISCAS)*, IEEE, 2013, pp. 477–480.
 - [61] A. Mercat, M. Viitanen, and J. Vanne, “Uvg dataset: 50/120fps 4k sequences for video codec analysis and development,” in *Proceedings of the 11th ACM multimedia systems conference*, 2020, pp. 297–302.
 - [62] H. Wang, W. Gan, S. Hu, J. Y. Lin, L. Jin, L. Song, P. Wang, I. Katsavounidis, A. Aaron, and C.-C. J. Kuo, “Mcl-jcv: A jnd-based h. 264/avc video quality assessment dataset,” in *2016 IEEE international conference on image processing (ICIP)*, IEEE, 2016, pp. 1509–1513.
 - [63] G. Bjontegaard, *Calculation of average PSNR differences between RD-curves*, ITU-T SG16/Q6, 13th VCEG Meeting, Austin, TX, USA, Doc. VCEG-M33, Apr. 2001.